Quantum Approximate Optimization Algorithm

Assessment of the QAOA algorithm (Max-Cut)

ANSWER BOOK

This book accompanies the separate *Question Paper* (80 questions, 3 hours, 240 marks). It contains

1. the **answer key**, section by section, with the topic tested by every question (page 2),
2. a **topic-wise map** grouping the questions by theme and a **quick-marking grid** (page 3), and
3. the **Answer Bank**: every question restated with the correct option marked ✓ and a complete solution with derivations (from page 4).

Marking scheme (every question)

Each question has **four** options (A), (B), (C) and (D). **ONLY ONE** of these four options is correct.

Full marks:	+3 if ONLY the correct option is chosen;
Zero marks:	0 if none of the options is chosen (question unanswered);
Negative marks:	−1 in all other cases (wrong option, or more than one option chosen).

How to evaluate

For each section count N_c = number of correct answers and N_w = number of questions answered wrongly (or with more than one option marked). Section marks = $3N_c - N_w$; unanswered questions score 0. The total is out of 240. Every numerical answer was re-computed independently (NumPy state vectors in the notebook's convention, brute force over all splits, and CUDA-Q for the QAOA circuits) by `verify_answers.py` in the source folder.

Conventions (identical to the Question Paper)

- Graph $G = (V, E)$ with nodes $0, 1, \ldots, n-1$ and edge weights w_{ij} ($w_{ij} = 1$ unless stated); $W = \sum_{(i,j)\in E} w_{ij}$. A split is a bitstring $z = z_0 z_1 \ldots z_{n-1}$ ($z_i \in \{0,1\}$ = group of node i, **node 0 written first**); $\bar{z}$ is z with every bit flipped. Spins: $s_i = (-1)^{z_i} = 1 - 2z_i$, the eigenvalue of Z_i on $|z\rangle$.
- Cut value $C(z) = \sum_{(i,j)\in E} w_{ij}\,[z_i \neq z_j] = \sum_{(i,j)} w_{ij}\frac{1-s_i s_j}{2}$; $C_{\max} = \max_z C(z)$.
- **Notebook convention** (`QAOA.ipynb`): $H_C = \sum_{(i,j)\in E} \frac{w_{ij}}{2} Z_i Z_j$, so $C = \frac{W}{2} - H_C$; $H_M = \sum_i X_i$; $|\boldsymbol{\gamma}, \boldsymbol{\beta}\rangle = e^{-i\beta_p H_M} e^{-i\gamma_p H_C} \ldots e^{-i\beta_1 H_M} e^{-i\gamma_1 H_C} |+\rangle^{\otimes n}$; $F_p(\boldsymbol{\gamma}, \boldsymbol{\beta}) = \langle \boldsymbol{\gamma}, \boldsymbol{\beta}|C|\boldsymbol{\gamma}, \boldsymbol{\beta}\rangle$; approximation ratio $r = F_p / C_{\max}$. The angle vector is `params` $= [\gamma_1, \ldots, \gamma_p, \beta_1, \ldots, \beta_p]$.
- Gates: $H = \frac{1}{\sqrt{2}}\left(\begin{smallmatrix} 1 & 1 \\ 1 & -1 \end{smallmatrix}\right)$; X, Y, Z are the Pauli matrices; $R_X(\theta) = e^{-i\theta X/2}$, $R_Z(\theta) = e^{-i\theta Z/2}$; CNOT$(c, t)$ flips qubit t when qubit c is 1. Two-qubit matrices use the basis order $|00\rangle, |01\rangle, |10\rangle, |11\rangle$ (lower-numbered qubit first); $|\pm\rangle = \frac{1}{\sqrt{2}}(|0\rangle \pm |1\rangle)$.
- **Notebook defaults:** random graph with 15 nodes and edge probability 0.3 (`SEED = 42`: 30 edges), $p = 2$, COBYLA with 80 evaluations, ramp initial angles, 4000 shots, exact objective via `cudaq.observe`. A *previously recorded run* means the outputs stored in the notebook (CUDA-Q 0.16.0, CPU target `qpp-cpu`).
- All measurements are in the computational basis; ln is the natural logarithm. Numerical answers are rounded.

Answer key

Answer distribution: (A) 20, (B) 20, (C) 20, (D) 20.

Section A — Mathematics and Graph Theory for QAOA

Q.	Ans.	Topic tested	Q.	Ans.	Topic tested
1	**A**	Bitstrings vs. partitions	11	**C**	Commuting exponentials
2	**A**	$C_{\max}$ of standard graphs	12	**A**	Shots for a target standard error
3	**B**	Random-split baseline $W/2$	13	**A**	Binomial statistics of P_{opt}
4	**B**	Cut as a QUBO polynomial	14	**D**	Complementary optimal strings
5	**D**	Random graph $G(n, q)$ statistics	15	**B**	QUBO $\to$ Ising conversion
6	**C**	Handshake lemma	16	**C**	Exact shift rule for derivatives
7	**C**	Odd cycles vs. odd node count	17	**D**	House graph: $C_{\max}$ and count
8	**B**	Complexity: max cut vs. min cut	18	**D**	House graph: random baseline
9	**A**	Spectrum of H_C on a triangle	19	**B**	Ground energy $= W/2 - C_{\max}$
10	**C**	Variance of a random cut	20	**D**	Local search: $\lvert E\rvert/2$, not optimal

Section B — Quantum Mechanics

Q.	Ans.	Topic tested	Q.	Ans.	Topic tested
21	**D**	Uniform state; QAOA enhancement	31	**B**	Conjugation $e^{i\beta X} Z e^{-i\beta X}$
22	**B**	Pauli products	32	**C**	State-vector memory on one GPU
23	**D**	Rotation-gate matrices	33	**C**	Bit-flip probability of R_X
24	**A**	Relative phase and interference	34	**B**	$\lvert+\rangle^{\otimes n}$ as top eigenstate of H_M
25	**D**	Expected cut from amplitudes	35	**B**	Complex-conjugation symmetry
26	**A**	Commutation of Pauli strings	36	**D**	Linearity of $\langle H_C\rangle$
27	**D**	Pauli propagation through CNOT	37	**A**	Mixer at $\beta = \pi/2$; period $\pi/2$
28	**A**	Entanglement by the cost gate	38	**A**	Measuring $Z_i Z_j$ from shots
29	**C**	Global phase vs. sign of γ	39	**C**	Trace: phase after the cost gate
30	**B**	$\langle Z_0 Z_1\rangle$ for a product state	40	**C**	Trace: cut probability after the mixer

Section C — QAOA Basics

Q.	Ans.	Topic tested	Q.	Ans.	Topic tested
41	**A**	`cost_hamiltonian()` output	51	**C**	Parameter-vector layout
42	**C**	All $\gamma_l = 0$	52	**D**	The ramp initialisation
43	**D**	CNOT–R_Z–CNOT decomposition	53	**A**	Minimise $\langle H_C\rangle$ = maximise $\langle C\rangle$
44	**D**	Why `rx(2*beta)`	54	**A**	Sign of the default start point
45	**B**	Gate and CNOT counts	55	**C**	Algorithm $\leftrightarrow$ code
46	**C**	Default circuit statistics	56	**B**	Triangle solved at $p = 1$
47	**D**	$p = 1$ single edge: formula, optimum	57	**B**	r vs. best sample
48	**B**	$p = 1$ on the ring	58	**C**	Best evaluated angles; no global guarantee
49	**A**	0.6924 bound; monotonicity in p	59	**A**	Path graph: $\langle Z_0 Z_1\rangle$
50	**D**	Limit $p \to \infty$	60	**B**	Path graph: $\langle C\rangle$, P_{opt}

Section D — Advanced QAOA and Implementation

Q.	Ans.	Topic tested	Q.	Ans.	Topic tested
61	**A**	Light cone of an edge	71	**C**	Depolarising-noise model
62	**B**	Ring at $p = 2$: 5/6	72	**C**	QAOA variants
63	**D**	Parameter shift vs. finite differences	73	**A**	INTERP initialisation
64	**D**	COBYLA `rhobeg`	74	**D**	Heuristic reference: upper bound on r
65	**C**	Shot noise of the objective	75	**C**	Recorded timing per gate
66	**B**	Brute-force memory	76	**B**	Run-time scaling to 25 nodes
67	**D**	Multi-GPU state vector	77	**A**	Quantum advantage: state of knowledge
68	**D**	Backend / method → target	78	**B**	Custom weighted edges
69	**C**	When MPS is efficient	79	**A**	Recorded run: partition probability
70	**A**	Shot-based objective on hardware	80	**B**	Recorded run: improvement over random

Topic-wise map of the questions

Every question appears under at least one theme; several appear under two.

Theme	Question numbers
Graphs, cuts and Max-Cut	1, 2, 6, 7, 14, 17, 18, 19, 20, 78
QUBO / Ising formulation and the cost Hamiltonian	4, 9, 15, 19, 29, 41, 53
Probability, statistics and sampling	3, 5, 10, 12, 13, 18, 21, 65, 79
Complexity and classical algorithms	8, 20, 49, 74, 77
Linear algebra: commutators, exponentials, Pauli algebra	11, 22, 23, 26, 27, 31, 37
Qubits, measurement, phases and entanglement	24, 25, 28, 30, 33, 34, 35, 36, 38, 39
Circuit construction and gate counts	43, 44, 45, 46, 55, 63
Initial state, mixer and parameters	34, 37, 42, 51, 52, 54, 73
Analysis at $p = 1$ and depth dependence	40, 47, 48, 49, 50, 56, 59, 60, 61, 62
Classical optimizers and gradients	16, 53, 58, 63, 64, 65, 73
Backends, memory and performance	32, 66, 67, 68, 69, 75, 76
Noise, hardware, variants and limitations	70, 71, 72, 77
Interpreting recorded results of `QAOA.ipynb`	13, 14, 21, 54, 57, 75, 79, 80

Quick-marking grid

1 **A**	2 **A**	3 **B**	4 **B**	5 **D**	6 **C**	7 **C**	8 **B**	9 **A**	10 **C**
11 **C**	12 **A**	13 **A**	14 **D**	15 **B**	16 **C**	17 **D**	18 **D**	19 **B**	20 **D**
21 **D**	22 **B**	23 **D**	24 **A**	25 **D**	26 **A**	27 **D**	28 **A**	29 **C**	30 **B**
31 **B**	32 **C**	33 **C**	34 **B**	35 **B**	36 **D**	37 **A**	38 **A**	39 **C**	40 **C**
41 **A**	42 **C**	43 **D**	44 **D**	45 **B**	46 **C**	47 **D**	48 **B**	49 **A**	50 **D**
51 **C**	52 **D**	53 **A**	54 **A**	55 **C**	56 **B**	57 **B**	58 **C**	59 **A**	60 **B**
61 **A**	62 **B**	63 **D**	64 **D**	65 **C**	66 **B**	67 **D**	68 **D**	69 **C**	70 **A**
71 **C**	72 **C**	73 **A**	74 **D**	75 **C**	76 **B**	77 **A**	78 **B**	79 **A**	80 **B**

SILICO FELLER

Answer Bank — questions with detailed solutions

SECTION A · Mathematics and Graph Theory for QAOA Q.1–20

Q.1 SINGLE-CORRECT MCQ · Bitstrings vs. partitions

A graph has $n = 15$ nodes. The number of bitstrings $z \in \{0,1\}^{15}$, and the number of *distinct* partitions $\{S, V \setminus S\}$ of the node set (unordered; $S = \varnothing$ allowed) that these bitstrings describe, are

(A) ✓ 32 768 and 16 384 **(B)** 16 384 and 8 192 **(C)** 32 768 and 16 383 **(D)** 32 768 and 32 768

Answer: (A) Detailed solution

Correct answer. **(A)** 32 768 and 16 384

Concepts required. A split is a bitstring z (z_i = group of node i). Relabelling the two groups (complementing z) gives the same partition and the same cut.

Relevant formulae. Number of bitstrings $= 2^n$; number of unordered partitions $= 2^n/2 = 2^{n-1}$ (pairs $\{z, \bar{z}\}$, $\bar{z}$ = all bits flipped).

Step-by-step explanation. Each node independently chooses group 0 or 1, so there are $2^{15} = 32\,768$ bitstrings. A bitstring z and its complement $\bar{z}$ (all bits flipped) describe the same unordered partition $\{S, V \setminus S\}$: the two groups only swap their labels. Complementation pairs the 2^{15} strings perfectly (no string equals its own complement), so the number of distinct partitions is $2^{15}/2 = 2^{14} = 16\,384$. The partition $\{\varnothing, V\}$ (strings $00\ldots0$ and $11\ldots1$, cut value 0) is one of them because $S = \varnothing$ is allowed.

Why the other options are incorrect.

(B) 16 384 bitstrings would be 2^{14}; each of the 15 nodes has 2 choices, so there are 2^{15} bitstrings.

(C) 16 383 excludes the partition $\{\varnothing, V\}$, but the question allows $S = \varnothing$ (cut value 0).

(D) 32 768 partitions counts every bitstring separately, forgetting that z and $\bar{z}$ describe the same partition.

Key takeaway. 2^n bitstrings, 2^{n-1} partitions: every cut appears twice in a QAOA histogram, as z and $\bar{z}$.

Q.2 MATCHING LISTS · $C_{\max}$ of standard graphs

Match each graph (List-I) with its maximum cut $C_{\max}$ (List-II). All edges have weight 1.

List-I		List-II	
(P)	cycle C_9 (9 nodes, 9 edges)	**(1)**	12
(Q)	complete graph K_5	**(2)**	9
(R)	complete bipartite graph $K_{3,3}$	**(3)**	6
(S)	Petersen graph (10 nodes, 15 edges, 3-regular)	**(4)**	8

(A) ✓ P→4, Q→3, R→2, S→1 **(B)** P→2, Q→3, R→1, S→4

(C) P→4, Q→2, R→3, S→1 **(D)** P→4, Q→3, R→1, S→2

Answer: (A) Detailed solution

Correct answer. **(A)** P→4, Q→3, R→2, S→1

Concepts required. $C_{\max} = |E|$ iff the graph is bipartite (no odd cycle). Each odd cycle forces at

least one uncut edge. For K_n only the group sizes matter.
Relevant formulae. $C_{\max}(C_n) = n$ (n even), $n-1$ (n odd); $C_{\max}(K_n) = \lfloor n/2 \rfloor \lceil n/2 \rceil$; $C_{\max}(K_{a,b}) = ab$.
Step-by-step explanation.

- C_9: an odd cycle cannot be 2-coloured, so at least one edge stays uncut; alternating colours around the cycle cuts all but one edge: $C_{\max} = 9 - 1 = 8$ (4).
- K_5: with k nodes on one side every one of the $k(5-k)$ cross pairs is an edge; the maximum over k is $2 \cdot 3 = 6$ (3). In general $C_{\max}(K_n) = \lfloor n/2 \rfloor \lceil n/2 \rceil$.
- $K_{3,3}$ is bipartite: putting the two natural sides in different groups cuts *all* 9 edges (2).
- Petersen graph: it contains 5-cycles (odd), so not all 15 edges can be cut. Every one of its 12 five-cycles needs an uncut edge, and the optimum leaves exactly 3 edges uncut: $C_{\max} = 12$ (1) (checked by brute force over all 2^{10} splits in `verify_answers.py`).

Hence P→4, Q→3, R→2, S→1.
Why the other options are incorrect.

(B) P→2 gives C_9 all 9 edges, as if it were bipartite — but an odd cycle always leaves one edge uncut; R→1 and S→4 then fail as above.

(C) Q→2 gives K_5 the value 9; the best split of K_5 is $2+3$ nodes, cutting $2 \cdot 3 = 6$ edges; and R→3 undercounts the bipartite $K_{3,3}$.

(D) R→1, S→2 gives $K_{3,3}$ the value 12 (it has only 9 edges) and the Petersen graph 9; the bipartite $K_{3,3}$ has all 9 edges cut, Petersen loses 3 of 15.

Key takeaway. Look for odd cycles first: they decide whether every edge can be cut.

Q.3 SINGLE-CORRECT MCQ · Random-split baseline $W/2$

A weighted graph has the edges $(0,1)$ with weight 3, $(1,2)$ with weight 1, $(2,3)$ with weight 2, $(3,0)$ with weight 4 and $(0,2)$ with weight 2. Every node is put into group 0 or group 1 independently with probability $\frac{1}{2}$. The expected cut value is

(A) 12 **(B)** ✓ 6 **(C)** 3 **(D)** 2.4

Answer: (B) **Detailed solution**

Correct answer. (B) 6
Concepts required. Linearity of expectation; for independent fair bits each edge is cut with probability $\frac{1}{2}$.
Relevant formulae. $\mathbb{E}[C] = \sum_{(i,j)\in E} w_{ij}\, P(z_i \neq z_j) = \frac{1}{2}\sum_{(i,j)} w_{ij} = \frac{W}{2}$, W = total weight.
Step-by-step explanation. Write $C(z) = \sum_{(i,j)\in E} w_{ij}\,[z_i \neq z_j]$. By linearity of expectation

$$\mathbb{E}[C] = \sum_{(i,j)} w_{ij}\, P(z_i \neq z_j).$$

For independent fair bits $P(z_i \neq z_j) = P(01) + P(10) = \frac{1}{4} + \frac{1}{4} = \frac{1}{2}$ for *every* edge, whatever the graph looks like. So $\mathbb{E}[C] = \frac{1}{2}W$ with $W = 3+1+2+4+2 = 12$, i.e. $\mathbb{E}[C] = 6$. This is exactly the "random-split baseline" $W/2$ printed by the notebook.
Why the other options are incorrect.

(A) 12 is W itself: it would require every edge to be cut with certainty.

(C) $3 = W/4$ uses $P(z_i \neq z_j) = \frac{1}{4}$, i.e. counts only the outcome 01 and forgets 10.

(D) 2.4 is the average weight $W/|E| = 12/5$, not an expected cut.

Key takeaway. The random-split baseline is always $W/2$, independent of the graph's structure.

Q.4 Single-correct MCQ · Cut as a QUBO polynomial

For the path graph $0-1-2$ (two unit edges), the cut value written as a polynomial in the binary variables $z_0, z_1, z_2 \in \{0,1\}$ is

(A) $z_0 + 2z_1 + z_2 - z_0z_1 - z_1z_2$ **(B)** ✓ $z_0 + 2z_1 + z_2 - 2z_0z_1 - 2z_1z_2$

(C) $z_0 + z_1 + z_2 - 2z_0z_1 - 2z_1z_2$ **(D)** $2 - z_0 - 2z_1 - z_2 + 2z_0z_1 + 2z_1z_2$

Answer: (B) Detailed solution

Correct answer. **(B)** $z_0 + 2z_1 + z_2 - 2z_0z_1 - 2z_1z_2$

Concepts required. XOR of two bits as a polynomial; QUBO = quadratic polynomial in binary variables.

Relevant formulae. $[a \neq b] = a \oplus b = a + b - 2ab$ for $a, b \in \{0,1\}$; $C(z) = \sum_{(i,j)} w_{ij}(z_i + z_j - 2z_iz_j)$.

Step-by-step explanation. For bits a, b: $[a \neq b] = a \oplus b = a + b - 2ab$ (check: $00 \to 0$, $01 \to 1$, $10 \to 1$, $11 \to 1 + 1 - 2 = 0$). Summing over the two edges:

$$C = (z_0 + z_1 - 2z_0z_1) + (z_1 + z_2 - 2z_1z_2) = z_0 + 2z_1 + z_2 - 2z_0z_1 - 2z_1z_2.$$

The middle node appears in both edges, hence the coefficient 2 of z_1 (in general the linear coefficient of z_i is the weighted degree of node i). Check $z = 010$: $C = 2$ (both edges cut) and the polynomial gives $0 + 2 + 0 - 0 - 0 = 2$. This is the QUBO form of Max-Cut.

Why the other options are incorrect.

(A) $-z_0z_1 - z_1z_2$ uses $a + b - ab$, which is OR, not XOR: for $z = 111$ it gives 2 instead of 0.

(C) $z_0 + z_1 + z_2 - \ldots$ forgets that node 1 belongs to two edges; for $z = 010$ it gives 1 instead of 2.

(D) $2 - z_0 - 2z_1 - \ldots$ is $|E| - C$, the number of *uncut* edges.

Key takeaway. The linear coefficient of z_i in the cut polynomial is the (weighted) degree of node i.

Q.5 Single-correct MCQ · Random graph $G(n, q)$ statistics

The notebook's default graph is a random graph on $n = 15$ nodes in which each of the possible edges is present independently with probability 0.3. The expected number of edges and its standard deviation are

(A) 45 and 3.97 **(B)** 31.5 and 22.05 **(C)** 63 and 6.64 **(D)** ✓ 31.5 and 4.70

Answer: (D) Detailed solution

Correct answer. **(D)** 31.5 and 4.70

Concepts required. Erdős–Rényi random graph $G(n, q)$: each of the $\binom{n}{2}$ pairs is an edge independently with probability q.

Relevant formulae. $|E| \sim \text{Binomial}(\binom{n}{2}, q)$: $\mathbb{E} = \binom{n}{2}q$, $\sigma = \sqrt{\binom{n}{2}q(1-q)}$; $\binom{15}{2} = 105$.

Step-by-step explanation. There are $\binom{15}{2} = \frac{15 \cdot 14}{2} = 105$ unordered node pairs, each an independent Bernoulli(0.3) trial. The edge count is Binomial(105, 0.3):

$$\mathbb{E}[|E|] = 105 \times 0.3 = 31.5, \qquad \text{Var} = 105 \times 0.3 \times 0.7 = 22.05, \qquad \sigma = \sqrt{22.05} \approx 4.70.$$

The instance actually generated with `SEED = 42` has 30 edges, well within one standard deviation (the notebook's text "about 31 edges" refers to this expectation; its printed output says 30).

Why the other options are incorrect.

(A) 45 and 3.97 use $\binom{10}{2} = 45$ pairs, i.e. the wrong number of nodes.

(B) 22.05 is the variance, not the standard deviation (take the square root: 4.70).

(C) 63 and 6.64 count ordered pairs ($15 \cdot 14 = 210$); an undirected edge is an unordered pair.

Key takeaway. Expected edges $= \binom{n}{2}q$; the actual seed-42 graph (30 edges) is a typical draw.

Q.6 Single-correct MCQ · Handshake lemma

The 15-node graph of the notebook has the degree sequence (nodes 0 to 14)

$$(d_0, \ldots, d_{14}) = (2, 3, 5, 4, 5, 4, 6, 1, 4, 3, 4, 2, 4, 6, 7).$$

The number of edges, and the number of nodes of odd degree, are

(A) 30 and 5 **(B)** 60 and 6 **(C)** ✓ 30 and 6 **(D)** 30 and 9

Answer: (C) Detailed solution

Correct answer. (C) 30 and 6
Concepts required. Handshake lemma; parity of the number of odd-degree nodes.
Relevant formulae. $\sum_{i \in V} d_i = 2|E|$; hence the number of odd-degree nodes is even.
Step-by-step explanation. **Handshake lemma:** every edge contributes 1 to the degree of each of its two end nodes, so $\sum_i d_i = 2|E|$. Here $\sum_i d_i = 2+3+5+4+5+4+6+1+4+3+4+2+4+6+7 = 60$, hence $|E| = 30$. Odd degrees: $3, 5, 5, 1, 3, 7$ (nodes 1, 2, 4, 7, 9, 14) — six nodes. The number of odd-degree nodes is always even, because $\sum_i d_i$ is even. (The degrees were recomputed from the edge list generated with `SEED = 42`.)
Why the other options are incorrect.
(A) Five odd-degree nodes is impossible: their count must be even (the six are nodes 1, 2, 4, 7, 9, 14).
(B) 60 is $\sum d_i$, which counts every edge twice.
(D) 9 is the number of *even*-degree nodes $(15 - 6)$.
Key takeaway. Divide the degree sum by two; odd degrees always come in pairs.

Q.7 Assertion–Reason · Odd cycles vs. odd node count

Assertion (A): The cycle C_7 (7 nodes, 7 unit edges) has $C_{\max} = 6$.
Reason (R): Every graph with an odd number of nodes has $C_{\max} < |E|$.

(A) Both Assertion (A) and Reason (R) are true, and R is the correct explanation of A
(B) Both A and R are true, but R is *not* the correct explanation of A
(C) ✓ A is true, but R is false
(D) A is false, but R is true

Answer: (C) Detailed solution

Correct answer. (C) A is true, but R is false
Concepts required. $C_{\max} = |E|$ exactly for bipartite graphs; a graph is bipartite iff it has no odd cycle.
Relevant formulae. $C_{\max}(C_n) = n - 1$ for odd n; bipartite $\iff$ no odd cycle $\iff$ $C_{\max} = |E|$.
Step-by-step explanation. **A is true:** $C_{\max} = |E|$ would require every edge to join the two groups, i.e. a proper 2-colouring; going round an odd cycle the colours must alternate an odd number of times and cannot close up. So at least one edge is uncut, and the alternating colouring 0101010 cuts exactly 6 edges. **R is false:** $C_{\max} = |E|$ holds exactly for *bipartite* graphs, and bipartiteness has nothing to do with the parity of the number of nodes. Counter-example: the path on 3 nodes, or the star with 7 nodes (one centre joined to six leaves) — both bipartite, both with $C_{\max} = |E|$. What matters is an *odd cycle*, not an odd node count. Answer: (C).
Why the other options are incorrect.
(A) needs A true, R true *and* R explaining A; R is false (an odd number of nodes does not prevent bipartiteness).

(B) needs A and R both true but R not explaining A; R is false.

(D) needs A false and R true; A is true (C_7 has $C_{\max} = 6$) and R is false.

Key takeaway. Odd *cycles*, not odd node counts, block a perfect cut.

Q.8 Statement I / II · Complexity: max cut vs. min cut

Statement I: A polynomial-time algorithm that solves Max-Cut *exactly* on every graph is known.

Statement II: Finding a *minimum* cut of a graph (both groups non-empty, non-negative weights) is NP-hard.

(A) Both Statement I and Statement II are true

(B) ✓ Both Statement I and Statement II are false

(C) Statement I is true, but Statement II is false

(D) Statement I is false, but Statement II is true

Answer: (B) Detailed solution

Correct answer. **(B)** Both Statement I and Statement II are false

Concepts required. Complexity: Max-Cut is NP-hard; minimum cut is polynomial.

Relevant formulae. Best polynomial-time approximation known: Goemans–Williamson, $\mathbb{E}[C] \geq 0.878\, C_{\max}$; global min cut: Stoer–Wagner $O(|V||E| + |V|^2 \log |V|)$.

Step-by-step explanation. **I is false:** Max-Cut is NP-hard (one of Karp's 21 NP-complete problems, decision version). No polynomial-time exact algorithm is known, and none exists unless P = NP. The best known polynomial-time guarantee is the Goemans–Williamson SDP rounding with expected ratio ≈ 0.878; exact solution in the notebook uses brute force over 2^n splits. **II is false:** the global minimum cut is polynomial — e.g. the Stoer–Wagner algorithm, or $n-1$ max-flow/min-cut computations. Maximising and minimising a cut have completely different complexity. Answer: (B).

Why the other options are incorrect.

(A) needs both statements true; neither statement is true.

(C) needs I true and II false; I is false: no exact polynomial algorithm for Max-Cut is known.

(D) needs I false and II true; II is false: min cut is solvable in polynomial time.

Key takeaway. Maximising a cut is hard, minimising it is easy — the direction matters.

Q.9 Single-correct MCQ · Spectrum of H_C on a triangle

For the triangle K_3 (unit weights) consider $H_C = \frac{1}{2}(Z_0Z_1 + Z_1Z_2 + Z_0Z_2)$. Its smallest eigenvalue and the number of computational basis states with that eigenvalue are

(A) ✓ $-\frac{1}{2}$, six basis states

(B) $-\frac{1}{2}$, three basis states

(C) $+\frac{3}{2}$, two basis states

(D) $-\frac{3}{2}$, two basis states

Answer: (A) Detailed solution

Correct answer. **(A)** $-\frac{1}{2}$, six basis states

Concepts required. H_C is diagonal; its value on $|z\rangle$ is computed from the spins $s_i = (-1)^{z_i}$; frustration.

Relevant formulae. $H_C(z) = \sum_{(i,j)} \frac{w_{ij}}{2} s_i s_j$; on a triangle $s_0s_1 \cdot s_1s_2 \cdot s_0s_2 = +1$; $C(z) = \frac{W}{2} - H_C(z)$.

Step-by-step explanation. H_C is diagonal: on $|z\rangle$ it equals $\frac{1}{2}(s_0s_1 + s_1s_2 + s_0s_2)$ with $s_i = (-1)^{z_i} = \pm 1$.

- $z = 000, 111$: all spins equal, every product $+1$: eigenvalue $\frac{3}{2}$ (2 states).
- one node differs from the other two (6 states such as $001, 010, \ldots$): products $-1, -1, +1$,

eigenvalue $-\frac{1}{2}$.

$-\frac{3}{2}$ would need all three products to be -1, but $(s_0s_1)(s_1s_2)(s_0s_2) = s_0^2s_1^2s_2^2 = +1$: impossible (the triangle is *frustrated*). So $E_0 = -\frac{1}{2}$ with 6 basis states, corresponding to $C = \frac{3}{2} - E_0 = 2 = C_{\max}$.

Why the other options are incorrect.

(B) Three counts partitions ($\{0\}|\{1,2\}$ etc.); each partition is two bitstrings, so six basis states.

(C) $+\frac{3}{2}$ is the *largest* eigenvalue (strings 000, 111, cut 0).

(D) $-\frac{3}{2}$ needs all three products $s_is_j = -1$, impossible because their product is $+1$ (frustration).

Key takeaway. Ground states of H_C = optimal cuts; on a frustrated triangle the ground energy is $-\frac{1}{2}$, six-fold.

Q.10 SINGLE-CORRECT MCQ · Variance of a random cut

For the notebook's 15-node graph (30 unit edges), a split z is chosen uniformly at random from all 2^{15} bitstrings. The variance of the cut value $C(z)$ is

(A) 15 **(B)** 3.75 **(C)** ✓ 7.5 **(D)** 30

Answer: (C) Detailed solution

Correct answer. (C) 7.5

Concepts required. Variance of a sum; pairwise independence of the edge indicators for a uniformly random split.

Relevant formulae. $X_e = z_i \oplus z_j \sim \text{Bernoulli}(\frac{1}{2})$, $\text{Var}(X_e) = \frac{1}{4}$, $\text{Cov}(X_e, X_f) = 0 \Rightarrow \text{Var}(C) = \sum_e w_e^2/4$ ($= |E|/4$ unweighted).

Step-by-step explanation. Write $C = \sum_e X_e$ with $X_e = [z_i \neq z_j]$ for edge $e = (i,j)$. Each X_e is Bernoulli($\frac{1}{2}$): $\text{Var}(X_e) = \frac{1}{2} \cdot \frac{1}{2} = \frac{1}{4}$. **Pairwise independence:** for two edges sharing a node, $e = (i,j)$, $f = (j,k)$: $X_e = z_i \oplus z_j$ and $X_f = z_j \oplus z_k$ are independent fair bits (given any z_j, z_i and z_k are independent and fair), so $\text{Cov}(X_e, X_f) = 0$; edges with no common node are trivially independent. Hence

$$\text{Var}(C) = \sum_e \text{Var}(X_e) = \frac{|E|}{4} = \frac{30}{4} = 7.5.$$

(Exact enumeration of the 2^{15} cut values of the actual graph gives 7.5.) The standard deviation is ≈ 2.74, so $C_{\max} = 25$ lies $(25 - 15)/2.74 \approx 3.7$ standard deviations above the random mean 15.

Why the other options are incorrect.

(A) 15 is the mean $W/2$, not the variance.

(B) 3.75 halves the correct value, e.g. by assuming that edges sharing a node are anticorrelated — they are uncorrelated.

(D) $30 = |E|$ uses variance 1 per edge (the variance of a ± 1 variable), but $X_e \in \{0,1\}$ has variance $\frac{1}{4}$.

Key takeaway. For a random split the cut has mean $|E|/2$ and variance $|E|/4$; good cuts are many standard deviations above the mean.

Q.11 Statement I / II · Commuting exponentials

Statement I: Because all the operators Z_iZ_j commute with one another, $e^{-i\gamma H_C} = \prod_{(i,j)\in E} e^{-i\gamma \frac{w_{ij}}{2} Z_iZ_j}$ holds exactly, with the factors in any order.

Statement II: Because the operators X_i also commute with one another, $e^{-i\beta H_M}\, e^{-i\gamma H_C} = e^{-i(\beta H_M + \gamma H_C)}$ holds for all β, γ.

(A) Both Statement I and Statement II are true

(B) Both Statement I and Statement II are false

(C) ✓ Statement I is true, but Statement II is false

(D) Statement I is false, but Statement II is true

Answer: (C) Detailed solution

Correct answer. **(C)** Statement I is true, but Statement II is false
Concepts required. $e^{A+B} = e^Ae^B$ when $[A,B]=0$; Baker–Campbell–Hausdorff otherwise.
Relevant formulae. $[Z_iZ_j, Z_kZ_l] = 0$; $\{X_i, Z_iZ_j\} = 0$ (anticommute); $e^Ae^B = e^{A+B+\frac{1}{2}[A,B]+\cdots}$.
Step-by-step explanation. For matrices, $e^{A+B} = e^Ae^B$ holds whenever $[A,B]=0$, but not in general. **I is true:** all Z_iZ_j are diagonal, hence commute, so the exponential of their sum factorises exactly into one two-qubit gate per edge, in any order. That is why the notebook can loop over the edges in list order. **II is false:** commuting *within* H_M only gives $e^{-i\beta H_M} = \prod_i e^{-i\beta X_i}$. The relevant commutator is $[H_M, H_C] \neq 0$ (e.g. X_i anticommutes with Z_iZ_j), so $e^{-i\beta H_M}e^{-i\gamma H_C} \neq e^{-i(\beta H_M+\gamma H_C)}$ in general (Baker–Campbell–Hausdorff). The non-commutativity is essential: if they commuted, QAOA could not move probability between bitstrings in a useful way. Answer: (C).
Why the other options are incorrect.
(A) needs both statements true; II is false.
(B) needs both statements false; I is true.
(D) needs I false and II true; I is true and II is false.
Key takeaway. Within a layer everything commutes; between cost and mixer layers nothing does — that is where QAOA's power lies.

Q.12 Single-correct MCQ · Shots for a target standard error

The expectation value $\langle Z_iZ_j\rangle$ is estimated as the average of the ±1 values $(-1)^{z_i \oplus z_j}$ over S independent shots. The smallest S that guarantees a standard error of at most 0.01 *for every state* is

(A) ✓ 10 000 **(B)** 2 500 **(C)** 100 **(D)** 40 000

Answer: (A) Detailed solution

Correct answer. **(A)** 10 000
Concepts required. Standard error of a sample mean; variance of a ±1 random variable.
Relevant formulae. $Y = \pm1$: $\mathrm{Var}(Y) = 1-\mu^2 \le 1$; $\mathrm{SE} = \sqrt{\mathrm{Var}(Y)/S} \le 1/\sqrt{S}$.
Step-by-step explanation. Each shot gives a random variable $Y = \pm1$ with mean $\mu = \langle Z_iZ_j\rangle$ and variance $\mathrm{Var}(Y) = \mathbb{E}[Y^2] - \mu^2 = 1-\mu^2 \le 1$ (worst case $\mu = 0$). The average of S independent shots has standard error $\sqrt{(1-\mu^2)/S} \le 1/\sqrt{S}$. Requiring $1/\sqrt{S} \le 0.01$ gives $S \ge 10^4$. (For the 0/1 indicator $[z_i \neq z_j]$ the variance is at most $\frac{1}{4}$, giving 2 500 — but that estimates $P(\text{cut})$, whose error is half the error of $\langle Z_iZ_j\rangle = 1 - 2P(\text{cut})$.)
Why the other options are incorrect.
(B) 2 500 uses the variance bound $\frac{1}{4}$ of a 0/1 variable; $\langle Z_iZ_j\rangle$ is an average of ±1 values, whose variance can be 1.
(C) 100 makes $1/S$ (not $1/\sqrt{S}$) equal to 0.01.

(D) 40 000 uses $(2/\varepsilon)^2$, confusing the width of the range $[-1, 1]$ with the standard deviation.
Key takeaway. Shot noise falls only like $1/\sqrt{S}$: one more decimal digit costs 100× more shots.

Q.13 MATCHING LISTS · Binomial statistics of P_{opt}

In the previously recorded notebook run, an optimal split is sampled with probability $P_{\text{opt}} = 0.06$ per shot. Treat the shots as independent and match each quantity (List-I) with its value (List-II).

List-I		List-II	
(P)	standard deviation of the number of optimal shots among 4000	**(1)**	75
(Q)	probability that 10 shots contain *no* optimal split	**(2)**	15.0
(R)	probability that 10 shots contain *at least one* optimal split	**(3)**	0.539
(S)	smallest number of shots giving $\geq 99\,\%$ probability of at least one optimal split	**(4)**	0.461

(A) ✓ P→2, Q→3, R→4, S→1
(B) P→2, Q→4, R→3, S→1
(C) P→1, Q→4, R→3, S→2
(D) P→1, Q→3, R→4, S→2

Answer: (A) Detailed solution

Correct answer. **(A)** P→2, Q→3, R→4, S→1
Concepts required. Binomial distribution of the number of successes in independent shots; complement rule.
Relevant formulae. $\sigma = \sqrt{SP(1-P)}$; $P(\text{none in } k) = (1-P)^k$; $S_{99\%} = \lceil \ln 0.01 / \ln(1-P) \rceil$.
Step-by-step explanation.

- P: the count is Binomial$(4000, 0.06)$, $\sigma = \sqrt{4000 \cdot 0.06 \cdot 0.94} = \sqrt{225.6} \approx 15.0$ (2). (Its mean is 240, exactly the number of optimal shots recorded.)
- Q: $P(\text{none in } 10) = 0.94^{10} \approx 0.539$ (3).
- R: complement: $1 - 0.539 = 0.461$ (4).
- S: need $1 - 0.94^S \geq 0.99 \iff 0.94^S \leq 0.01 \iff S \geq \dfrac{\ln 0.01}{\ln 0.94} = \dfrac{-4.605}{-0.0619} \approx 74.4$, so $S = 75$ (1).

Hence P→2, Q→3, R→4, S→1.
Why the other options are incorrect.

(B) Q→4, R→3 swaps an event and its complement: *no* optimal split in 10 shots has probability $0.94^{10} = 0.539$.

(C) P→1 and S→2 are wrong as in the previous option, and Q/R are also swapped.

(D) P→1, S→2 swaps the standard deviation and the shot count: with 15 shots the success probability is only $1 - 0.94^{15} = 0.60$.

Key takeaway. With $P_{\text{opt}} = 0.06$, about 75 shots already give 99 % confidence of seeing an optimum.

Q.14 SINGLE-CORRECT MCQ · Complementary optimal strings

The notebook's brute-force search reports four optimal bitstrings (cut value 25):

`001101110100010`, `001101110110010`, `110010001001101`, `110010001011101`.

The number of distinct optimal partitions, and the bitstring describing the same partition as `001101110100010`, are

(A) 2; `110010001001101` **(B)** 2; `010001011101100`

(C) 4; `110010001011101` **(D)** ✓ 2; `110010001011101`

Answer: (D) Detailed solution

Correct answer. **(D)** 2; `110010001011101`
Concepts required. $C(z) = C(\bar{z})$: optimal strings come in complementary pairs.
Relevant formulae. $\bar{z}_i = 1 - z_i$; $[z_i \neq z_j] = [\bar{z}_i \neq \bar{z}_j]$; number of optimal partitions = (number of optimal strings)/2.
Step-by-step explanation. $C(z) = C(\bar{z})$: flipping every bit swaps the group labels and keeps every edge cut or uncut. So optimal strings come in complementary pairs, and each pair is one partition. Complement of `001101110100010`: flip each bit ⇒ `110010001011101`, which is in the list. Complement of `001101110110010`: `110010001001101`, also in the list. So the 4 strings form 2 pairs: **2 distinct optimal partitions**. (The two partitions differ only in node 10, which has as many neighbours on each side when the rest is fixed.)
Why the other options are incorrect.

(A) `110010001001101` is the complement of the *other* string `001101110110010` (they differ in bit 10).

(B) `010001011101100` is the string written backwards (reversal), not the complement.

(C) 4 counts bitstrings, not partitions.

Key takeaway. Complement = flip every bit; reversal is a different operation and does not preserve cuts.

Q.15 SINGLE-CORRECT MCQ · QUBO → Ising conversion

The QUBO function $f(x_0, x_1) = 2x_0x_1 - x_0 - x_1$ with $x_i \in \{0, 1\}$ is rewritten with spins via $x_i = (1 - s_i)/2$, $s_i \in \{+1, -1\}$. The result is

(A) $\frac{1}{2}\, s_0 s_1 - 1 + \frac{1}{2}(s_0 + s_1)$ **(B)** ✓ $\frac{1}{2}\,(s_0 s_1 - 1)$

(C) $\frac{1}{4}\,(s_0 s_1 - 1)$ **(D)** $\frac{1}{2}\,(1 - s_0 s_1)$

Answer: (B) Detailed solution

Correct answer. **(B)** $\frac{1}{2}\,(s_0 s_1 - 1)$
Concepts required. Change of variables between binary (QUBO) and spin (Ising) form.
Relevant formulae. $x_i = \frac{1-s_i}{2}$, $x_i x_j = \frac{1}{4}(1 - s_i - s_j + s_i s_j)$, $[x_i \neq x_j] = \frac{1}{2}(1 - s_i s_j)$.
Step-by-step explanation. Substitute: $x_0 x_1 = \frac{1}{4}(1 - s_0)(1 - s_1) = \frac{1}{4}(1 - s_0 - s_1 + s_0 s_1)$, so $2x_0 x_1 = \frac{1}{2}(1 - s_0 - s_1 + s_0 s_1)$; $-x_0 - x_1 = -\frac{1}{2}(1 - s_0) - \frac{1}{2}(1 - s_1) = -1 + \frac{1}{2}(s_0 + s_1)$. Adding, the linear terms cancel:

$$f = \tfrac{1}{2} - \tfrac{1}{2}(s_0 + s_1) + \tfrac{1}{2} s_0 s_1 - 1 + \tfrac{1}{2}(s_0 + s_1) = \tfrac{1}{2}(s_0 s_1 - 1).$$

Check: $x = 01$: $f = -1$ and $s = (+1, -1)$ gives $\frac{1}{2}(-1 - 1) = -1$. So $f = -[x_0 \neq x_1]$: minimising f is Max-Cut on one edge, and its Ising form is (up to a constant) $\frac{1}{2} Z_0 Z_1$ — one term of H_C.
Why the other options are incorrect.

(A) The linear terms do not survive: $-\frac{1}{2}(s_0 + s_1)$ from $2x_0x_1$ cancels $+\frac{1}{2}(s_0 + s_1)$ from $-x_0 - x_1$.

(C) $\frac{1}{4}(s_0 s_1 - 1)$ forgets the factor 2 of $2x_0x_1$; it gives $f(0, 1) = -\frac{1}{2}$.

(D) $\frac{1}{2}(1 - s_0 s_1)$ has the wrong overall sign: it equals $+[x_0 \neq x_1]$, while $f(0,1) = -1$.
Key takeaway. Max-Cut is a QUBO whose Ising form has only $Z_i Z_j$ terms (no fields): exactly the shape of H_C.

Q.16 SINGLE-CORRECT MCQ · Exact shift rule for derivatives

Let $f(\theta) = a + b\cos\theta + c\sin\theta$ with constants a, b, c. Which expression equals $f'(\theta)$ *exactly* for every θ?

(A) $\frac{1}{2}[f(\theta + \frac{\pi}{4}) - f(\theta - \frac{\pi}{4})]$
(B) $f(\theta + \frac{\pi}{2}) - f(\theta)$
(C) ✓ $\frac{1}{2}[f(\theta + \frac{\pi}{2}) - f(\theta - \frac{\pi}{2})]$
(D) $\frac{1}{\pi}[f(\theta + \frac{\pi}{2}) - f(\theta - \frac{\pi}{2})]$

Answer: (C) Detailed solution

Correct answer. (C) $\frac{1}{2}[f(\theta + \frac{\pi}{2}) - f(\theta - \frac{\pi}{2})]$
Concepts required. Trigonometric shift identities; expectation values are trigonometric in a single gate angle.
Relevant formulae. $\cos(\theta \pm \frac{\pi}{2}) = \mp\sin\theta$, $\sin(\theta \pm \frac{\pi}{2}) = \pm\cos\theta$; parameter shift $f' = \frac{1}{2}[f(\theta + \frac{\pi}{2}) - f(\theta - \frac{\pi}{2})]$.
Step-by-step explanation. $f'(\theta) = -b\sin\theta + c\cos\theta$. Using $\cos(\theta \pm \frac{\pi}{2}) = \mp\sin\theta$ and $\sin(\theta \pm \frac{\pi}{2}) = \pm\cos\theta$:

$$f(\theta + \tfrac{\pi}{2}) - f(\theta - \tfrac{\pi}{2}) = b(-\sin\theta - \sin\theta) + c(\cos\theta + \cos\theta) = 2(-b\sin\theta + c\cos\theta) = 2f'(\theta).$$

So $f'(\theta) = \frac{1}{2}[f(\theta + \frac{\pi}{2}) - f(\theta - \frac{\pi}{2})]$ exactly — not an approximation. This is the *parameter-shift rule*: an expectation value depends on the angle of one gate $e^{-i\theta P/2}$ (P a Pauli operator) exactly in this form, so two circuit runs give the exact derivative.
Why the other options are incorrect.
(A) A shift of $\pi/4$ gives $\frac{1}{\sqrt{2}}f'$; the factor $\frac{1}{2}$ is right only for shifts of $\pm\pi/2$.
(B) $f(\theta + \frac{\pi}{2}) - f(\theta) = -b\sin\theta + c\cos\theta - b\cos\theta - c\sin\theta \neq f'$ in general.
(D) Dividing by π treats it as a central finite difference with step $\pi/2$; it gives $\frac{2}{\pi}f' \neq f'$.
Key takeaway. Large shifts, exact derivative: two circuit runs per gate angle.

Paragraph for Questions 17 and 18

The "house graph" has the 5 nodes $0, 1, 2, 3, 4$ and the 6 unit-weight edges $(0,1), (1,2), (2,3), (3,0)$ (a square) and $(0,4), (1,4)$ (a roof: node 4 forms a triangle with nodes 0 and 1). A split is a bitstring $z = z_0 z_1 z_2 z_3 z_4$; its cut value $C(z)$ counts the edges whose two end nodes are in different groups.

Q.17 PARAGRAPH BASED · House graph: $C_{\max}$ and count

The maximum cut $C_{\max}$ of the house graph, and the number of bitstrings z that achieve it, are

(A) 5 and 6
(B) 5 and 2
(C) 6 and 2
(D) ✓ 5 and 4

Answer: (D) Detailed solution

Correct answer. (D) 5 and 4
Concepts required. Upper bound from an odd cycle; construction of a matching lower bound; counting.
Relevant formulae. Each odd cycle has at least one uncut edge; a bipartite subgraph can be cut completely.
Step-by-step explanation. **Upper bound:** the triangle $0, 1, 4$ is an odd cycle, so at least one of its three edges is uncut: $C \leq 6 - 1 = 5$. **Achieving 5:** the square $0{-}1{-}2{-}3$ is bipartite; put

$\{0,2\}$ and $\{1,3\}$ in different groups (all 4 square edges cut). Node 4 is joined to 0 and 1, which are now in different groups, so exactly one roof edge is cut *whichever* group node 4 joins: total $4+1=5$. **Counting:** a cut of 5 must cut all four square edges (if a square edge were uncut, at most $3+2=5$ with both roof edges cut, which needs $z_4 \neq z_0$ and $z_4 \neq z_1$, so $z_0 = z_1$ — but then edge $(0,1)$ is uncut too, giving at most 4). So $z_0z_1z_2z_3 \in \{0101, 1010\}$ and z_4 is free: $2 \times 2 = 4$ strings: $01010, 01011, 10100, 10101$.

Why the other options are incorrect.

(A) Six strings would require more optimal patterns of the square; only 0101 and 1010 cut all four square edges.

(B) 2 counts partitions ($\{0,2,4\}|\{1,3\}$ and $\{0,2\}|\{1,3,4\}$) instead of bitstrings.

(C) 6 would cut every edge, impossible because of the triangle $0,1,4$.

Key takeaway. Bound with odd cycles, then construct; node 4 is free because its two neighbours are always on opposite sides.

Q.18 PARAGRAPH BASED · House graph: random baseline

A split of the house graph is drawn uniformly at random. The probability that it is optimal, and the ratio $\mathbb{E}[C]/C_{\max}$ of the random split, are

(A) $\frac{1}{16}$ and 0.6 **(B)** $\frac{1}{4}$ and 0.6 **(C)** $\frac{1}{8}$ and 0.5 **(D)** ✓ $\frac{1}{8}$ and 0.6

Answer: (D) Detailed solution

Correct answer. (D) $\frac{1}{8}$ and 0.6

Concepts required. Uniform distribution over 2^n bitstrings; random baseline $W/2$.

Relevant formulae. $P(\text{optimal}) = \#\text{optimal strings}/2^n$; $r_{\text{random}} = (W/2)/C_{\max}$.

Step-by-step explanation. There are $2^5 = 32$ equally likely bitstrings and 4 are optimal (previous question): $P = \frac{4}{32} = \frac{1}{8}$. The expected cut of a uniform split is $W/2 = 6/2 = 3$ (every edge is cut with probability $\frac{1}{2}$), so the random approximation ratio is $3/5 = 0.6$. Any useful algorithm — QAOA included — must beat $r = 0.6$ on this graph.

Why the other options are incorrect.

(A) $\frac{1}{16}$ counts 2 optimal strings (partitions) instead of 4.

(B) $\frac{1}{4}$ divides by $16 = 2^4$, forgetting one node.

(C) 0.5 divides $W/2 = 3$ by $|E| = 6$ instead of by $C_{\max} = 5$.

Key takeaway. Random guessing already achieves $r = (W/2)/C_{\max}$, here 0.6.

Q.19 SINGLE-CORRECT MCQ · Ground energy $= W/2 - C_{\max}$

For the Petersen graph (10 nodes, 15 unit edges, $C_{\max} = 12$), the smallest eigenvalue of $H_C = \sum_{(i,j)\in E} \frac{1}{2} Z_i Z_j$ is

(A) -7.5 **(B)** ✓ -4.5 **(C)** -12 **(D)** $+4.5$

Answer: (B) Detailed solution

Correct answer. (B) -4.5

Concepts required. Relation between the cut and the Ising energy on basis states.

Relevant formulae. $C(z) = \frac{W}{2} - H_C(z) \Rightarrow E_0(H_C) = \frac{W}{2} - C_{\max}$.

Step-by-step explanation. On a basis state $|z\rangle$, $C(z) = \frac{W}{2} - H_C(z)$ with $W = 15$, because $[z_i \neq z_j] = \frac{1}{2}(1 - s_i s_j)$ and summing gives $C = \frac{|E|}{2} - \frac{1}{2}\sum s_i s_j$. Minimising H_C is maximising C: $E_0 = \frac{W}{2} - C_{\max} = 7.5 - 12 = -4.5$. -7.5 would require all 15 products $s_i s_j = -1$ (every edge cut), impossible because the graph has odd cycles.

Why the other options are incorrect.

(A) -7.5 assumes all 15 edges can be cut; the Petersen graph has 5-cycles.

(C) -12 confuses the energy with the cut value; the shift by $W/2$ is missing.

(D) $+4.5$ has the wrong sign: large cuts correspond to *low* energy.

Key takeaway. Ground energy $= W/2 - C_{\max}$: minimising H_C and maximising C are the same problem.

Q.20 Assertion–Reason · Local search: $|E|/2$, not optimal

The notebook's `local_search_maxcut` repeatedly flips single nodes while this strictly increases the cut (unit weights), starting from random splits.

Assertion (A): Every split returned by one run of this greedy procedure is an optimal split.

Reason (R): When no single flip increases the cut, every node has at least half of its incident edges cut.

(A) Both Assertion (A) and Reason (R) are true, and R is the correct explanation of A

(B) Both A and R are true, but R is *not* the correct explanation of A

(C) A is true, but R is false

(D) ✓ A is false, but R is true

Answer: (D) Detailed solution

Correct answer. (D) A is false, but R is true

Concepts required. Local versus global optimum; the $|E|/2$ guarantee of single-flip local search.

Relevant formulae. Flip gain at node v: $\Delta C = u_v - c_v$; local optimum $\Rightarrow c_v \geq d_v/2 \Rightarrow C \geq |E|/2$.

Step-by-step explanation. **R is true:** flipping node v turns its cut edges into uncut ones and vice versa, so the cut changes by $u_v - c_v$ (u_v, c_v = numbers of uncut and cut edges at v). At a local optimum no flip strictly improves, so $u_v - c_v \leq 0$, i.e. $c_v \geq d_v/2$. Summing over v (each cut edge counted twice): $2C \geq \sum_v d_v/2 = |E|$, so a local optimum always has $C \geq |E|/2$. **A is false:** a local optimum need not be global — greedy search can get stuck. That is exactly why the notebook restarts it 200 times and labels its result "heuristic local search (best known, NOT guaranteed optimal)". Answer: (D).

Why the other options are incorrect.

(A) needs A true, R true *and* R explaining A; A is false: a local optimum need not be global.

(B) needs A and R both true but R not explaining A; A is false.

(C) needs A true and R false; A is false and R is true.

Key takeaway. Greedy local search guarantees only $|E|/2$; restarts help but prove nothing.

SECTION B · Quantum Mechanics Q.21–40

Q.21 Single-correct MCQ · Uniform state; QAOA enhancement

The 15-qubit state $|+\rangle^{\otimes 15}$ is measured in the computational basis. The notebook's graph has 4 optimal bitstrings. The probability of obtaining an optimal bitstring, and the factor by which the previously recorded QAOA value $P_{\text{opt}} = 0.06$ exceeds it, are

(A) 6.10×10^{-5}; about 980
(B) 3.05×10^{-5}; about 1970
(C) 2.44×10^{-4}; about 245
(D) ✓ 1.22×10^{-4}; about 490

Answer: (D) Detailed solution

Correct answer. (D) 1.22×10^{-4}; about 490
Concepts required. Born rule; uniform superposition; probability of a set of outcomes.
Relevant formulae. $|+\rangle^{\otimes n} = 2^{-n/2}\sum_z |z\rangle$, $P(z) = 2^{-n}$; $P(\text{set}) = |\text{set}|/2^n$.
Step-by-step explanation. $|+\rangle^{\otimes n} = 2^{-n/2}\sum_z |z\rangle$: every one of the $2^{15} = 32\,768$ bitstrings has amplitude $2^{-15/2}$ and probability $|2^{-15/2}|^2 = 1/32\,768$ (Born rule). Four optimal strings: $P = 4/32\,768 = 1.22 \times 10^{-4}$. Enhancement: $0.06/(1.22 \times 10^{-4}) = 0.06 \times 8192 \approx 491.5$. QAOA concentrates probability on good cuts; the initial uniform superposition is simply random guessing.
Why the other options are incorrect.
(A) 6.10×10^{-5} counts 2 partitions as 2 bitstrings; there are 4 optimal bitstrings.
(B) $3.05 \times 10^{-5} = 1/32\,768$ counts only one optimal string, not four.
(C) 2.44×10^{-4} divides by 2^{14} (partitions); measurements return bitstrings, of which there are 2^{15}.
Key takeaway. $|+\rangle^{\otimes n}$ is random guessing; QAOA's job is to raise P_{opt} far above $|\text{opt}|/2^n$.

Q.22 Single-correct MCQ · Pauli products

The two-qubit operator $(Z \otimes Z)(X \otimes X)$ equals

(A) $iY \otimes Y$
(B) ✓ $-Y \otimes Y$
(C) $Y \otimes Y$
(D) $-iY \otimes Y$

Answer: (B) Detailed solution

Correct answer. (B) $-Y \otimes Y$
Concepts required. Pauli multiplication table; tensor products multiply factor-wise.
Relevant formulae. $ZX = iY$, $XZ = -iY$, $XY = iZ$, $YZ = iX$; $(A \otimes B)(C \otimes D) = AC \otimes BD$.
Step-by-step explanation. Tensor products multiply factor by factor: $(Z\otimes Z)(X\otimes X) = (ZX)\otimes(ZX)$. Single-qubit Pauli algebra: $ZX = iY$ (check: $ZX = \left(\begin{smallmatrix}1&0\\0&-1\end{smallmatrix}\right)\left(\begin{smallmatrix}0&1\\1&0\end{smallmatrix}\right) = \left(\begin{smallmatrix}0&1\\-1&0\end{smallmatrix}\right) = i\left(\begin{smallmatrix}0&-i\\i&0\end{smallmatrix}\right) = iY$). So $(iY)\otimes(iY) = i^2\, Y\otimes Y = -Y\otimes Y$. Also $(X\otimes X)(Z\otimes Z) = (XZ)\otimes(XZ) = (-iY)\otimes(-iY) = -Y\otimes Y$: the same — $Z\otimes Z$ and $X\otimes X$ commute, although Z and X anticommute.
Why the other options are incorrect.
(A) $iY \otimes Y$ keeps only one factor i (two qubits give $i \cdot i$).
(C) $Y \otimes Y$ forgets the factor $i^2 = -1$.
(D) $-iY \otimes Y$ mixes $ZX = iY$ on one qubit with $XZ = -iY$ on the other; both factors are ZX.
Key takeaway. Count the phases qubit by qubit; two anticommuting positions make the products commute.

Q.23 MATCHING LISTS · Rotation-gate matrices

Match each operator (List-I) with its matrix (List-II). Two-qubit matrices use the basis order $|00\rangle, |01\rangle, |10\rangle, |11\rangle$.

List-I		List-II	
(P)	$R_X(2\beta)$	**(1)**	$\mathrm{diag}\,(e^{-i\theta/2}, e^{i\theta/2}, e^{i\theta/2}, e^{-i\theta/2})$
(Q)	$R_Z(\theta)$	**(2)**	$\begin{pmatrix} \cos\beta & -i\sin\beta \\ -i\sin\beta & \cos\beta \end{pmatrix}$
(R)	$e^{-i\theta\, Z\otimes Z/2}$	**(3)**	$\frac{1}{\sqrt{2}}\begin{pmatrix} 1 & 1 \\ 1 & -1 \end{pmatrix}$
(S)	H	**(4)**	$\mathrm{diag}\,(e^{-i\theta/2}, e^{i\theta/2})$

(A) P→3, Q→4, R→1, S→2 **(B)** P→2, Q→1, R→4, S→3

(C) P→4, Q→2, R→1, S→3 **(D)** ✓ P→2, Q→4, R→1, S→3

Answer: (D) Detailed solution

Correct answer. **(D)** P→2, Q→4, R→1, S→3

Concepts required. Exponentials of Pauli operators; half-angle convention of rotation gates.

Relevant formulae. $e^{-i\phi P} = \cos\phi\, I - i\sin\phi\, P$ $(P^2 = I)$; $R_X(\theta) = e^{-i\theta X/2}$, $R_Z(\theta) = e^{-i\theta Z/2}$; $Z\otimes Z = \mathrm{diag}(1,-1,-1,1)$.

Step-by-step explanation. For a Pauli P (so $P^2 = I$): $e^{-i\phi P} = \cos\phi\, I - i\sin\phi\, P$.

- P: $R_X(2\beta) = e^{-i\beta X} = \cos\beta\, I - i\sin\beta\, X$ (2).
- Q: $R_Z(\theta) = e^{-i\theta Z/2} = \mathrm{diag}(e^{-i\theta/2}, e^{+i\theta/2})$ (4).
- R: $Z\otimes Z = \mathrm{diag}(+1,-1,-1,+1)$, so $e^{-i\theta Z\otimes Z/2} = \mathrm{diag}(e^{-i\theta/2}, e^{i\theta/2}, e^{i\theta/2}, e^{-i\theta/2})$ (1): equal bits get $e^{-i\theta/2}$, different bits get $e^{+i\theta/2}$.
- S: Hadamard (3).

Why the other options are incorrect.

(A) P→3 assigns H to $R_X(2\beta)$; no value of β gives H (e.g. $\beta = \pi/4$ gives $\frac{1}{\sqrt{2}}\left(\begin{smallmatrix} 1 & -i \\ -i & 1 \end{smallmatrix}\right)$).

(B) Q→1, R→4 swaps the one-qubit R_Z (2×2) and the two-qubit ZZ rotation (4×4).

(C) P→4, Q→2 swaps R_X and R_Z: an X rotation has off-diagonal entries, a Z rotation is diagonal.

Key takeaway. Z-type rotations are diagonal (phases), X-type rotations mix $|0\rangle$ and $|1\rangle$.

Q.24 SINGLE-CORRECT MCQ · Relative phase and interference

The state $R_Z(\theta)\,|+\rangle$ is passed through a Hadamard gate and then measured in the computational basis. The probability of the outcome 0 is

(A) ✓ $\cos^2(\theta/2)$ **(B)** $\sin^2(\theta/2)$ **(C)** $\cos^2\theta$ **(D)** $\frac{1}{2}$

Answer: (A) Detailed solution

Correct answer. **(A)** $\cos^2(\theta/2)$

Concepts required. Relative phase is invisible in the Z basis but becomes a probability after a Hadamard (interference).

Relevant formulae. $H\frac{1}{\sqrt{2}}(|0\rangle + e^{i\theta}|1\rangle) = \frac{1}{2}[(1+e^{i\theta})|0\rangle + (1-e^{i\theta})|1\rangle]$; $|1+e^{i\theta}|^2 = 4\cos^2\frac{\theta}{2}$.

Step-by-step explanation. $R_Z(\theta)|+\rangle = \frac{1}{\sqrt{2}}(e^{-i\theta/2}|0\rangle + e^{i\theta/2}|1\rangle) = e^{-i\theta/2}\frac{1}{\sqrt{2}}(|0\rangle + e^{i\theta}|1\rangle)$. Apply H: $\frac{1}{\sqrt{2}}(|0\rangle + e^{i\theta}|1\rangle) \mapsto \frac{1}{2}[(1+e^{i\theta})|0\rangle + (1-e^{i\theta})|1\rangle]$. $P(0) = \frac{1}{4}|1+e^{i\theta}|^2 = \frac{1}{4}(2+2\cos\theta) = \cos^2(\theta/2)$.

Without the Hadamard the result would be $\frac{1}{2}$ for every θ: a Z-rotation only changes a *relative phase*, which becomes visible only after interference. In QAOA the mixer plays this interfering role for the cost phases.

Why the other options are incorrect.

(B) $\sin^2(\theta/2)$ is $P(1)$, not $P(0)$.

(C) $\cos^2\theta$ doubles the angle: the relative phase is θ, and the probability is $\cos^2(\theta/2)$.

(D) $\frac{1}{2}$ is the result *without* the Hadamard.

Key takeaway. Phases need interference to be seen — in QAOA the mixer does this.

Q.25 Single-correct MCQ · Expected cut from amplitudes

For one unit edge $(0,1)$ the qubits are in the state $|\psi\rangle = \frac{1}{2}|00\rangle + \frac{\sqrt{3}}{2}|01\rangle$. The expected cut value $\langle C\rangle = \langle \frac{1}{2}(1 - Z_0Z_1)\rangle$ is

(A) $\frac{1}{4}$ **(B)** $-\frac{1}{2}$ **(C)** $\frac{\sqrt{3}}{2}$ **(D)** ✓ $\frac{3}{4}$

Answer: (D) Detailed solution

Correct answer. **(D)** $\frac{3}{4}$

Concepts required. Expectation of a diagonal observable from the Born probabilities.

Relevant formulae. $\langle Z_0Z_1\rangle = \sum_z P(z)\, s_0 s_1$; $\langle C_{01}\rangle = \frac{1}{2}(1 - \langle Z_0Z_1\rangle) = P(z_0 \neq z_1)$.

Step-by-step explanation. Z_0Z_1 is diagonal with eigenvalue $+1$ on $|00\rangle$ and -1 on $|01\rangle$. Born rule: $P(00) = \frac{1}{4}$, $P(01) = \frac{3}{4}$. $\langle Z_0Z_1\rangle = \frac{1}{4}(+1) + \frac{3}{4}(-1) = -\frac{1}{2}$, so $\langle C\rangle = \frac{1}{2}(1 - (-\frac{1}{2})) = \frac{3}{4}$. Directly: the edge is cut exactly in outcome 01, which has probability $\frac{3}{4}$.

Why the other options are incorrect.

(A) $\frac{1}{4}$ is $P(00)$, the probability that the edge is *not* cut.

(B) $-\frac{1}{2}$ is $\langle Z_0Z_1\rangle$, not the cut.

(C) $\frac{\sqrt{3}}{2}$ is an amplitude; probabilities are squared moduli.

Key takeaway. $\langle C\rangle$ for an edge is just the probability that the edge is cut.

Q.26 Single-correct MCQ · Commutation of Pauli strings

Which of the following operators commutes with Z_0Z_1 (on two or more qubits)?

(A) ✓ X_0X_1 **(B)** Y_0Z_1 **(C)** X_0 **(D)** X_1

Answer: (A) Detailed solution

Correct answer. **(A)** X_0X_1

Concepts required. Commutation of Pauli strings: count anticommuting positions.

Relevant formulae. $PQ = (-1)^k QP$, k = number of positions where the single-qubit factors anticommute.

Step-by-step explanation. For Pauli products, count the positions where the two factors anti-commute: an *even* count means the products commute, odd means they anticommute. X_0X_1 vs Z_0Z_1: positions 0 (X vs Z, anticommute) and 1 (X vs Z, anticommute) — two, so they commute. X_0: one anticommuting position. Y_0Z_1: position 0 anticommutes (Y vs Z), position 1 commutes — one. X_1: one. So only X_0X_1 commutes with Z_0Z_1 (consistent with Q.22).

Why the other options are incorrect.

(B) Y_0Z_1: Y_0 vs Z_0 anticommute, Z_1 vs Z_1 commute — one position, so they anticommute.

(C) X_0 anticommutes with Z_0 at one position.

(D) X_1 anticommutes with Z_1 at one position.

Key takeaway. Even number of clashes ⇒ commute; this is why the mixer does not commute with the cost terms.

Q.27 MATCHING LISTS · Pauli propagation through CNOT

Conjugation by a CNOT with control qubit 0 and target qubit 1 maps a Pauli operator P to $\mathrm{CNOT}\,P\,\mathrm{CNOT}$. Match each P (List-I) with its image (List-II).

List-I		List-II	
(P)	$X \otimes I$	**(1)**	$Z \otimes Z$
(Q)	$I \otimes X$	**(2)**	$X \otimes X$
(R)	$Z \otimes I$	**(3)**	$I \otimes X$
(S)	$I \otimes Z$	**(4)**	$Z \otimes I$

(A) P→3, Q→2, R→1, S→4
(B) P→2, Q→3, R→1, S→4
(C) P→3, Q→2, R→4, S→1
(D) ✓ P→2, Q→3, R→4, S→1

Answer: (D) Detailed solution

Correct answer. (D) P→2, Q→3, R→4, S→1
Concepts required. Propagation of Pauli operators through a CNOT (Clifford conjugation).
Relevant formulae. $\mathrm{CNOT}_{0\to1}$: $X_0 \to X_0X_1$, $X_1 \to X_1$, $Z_0 \to Z_0$, $Z_1 \to Z_0Z_1$.
Step-by-step explanation. Rules ($\mathrm{CNOT}^2 = I$, so conjugation is $P \mapsto \mathrm{CNOT}\,P\,\mathrm{CNOT}$):

- X on the control *spreads* to the target: $X \otimes I \to X \otimes X$ (2). (Flipping the control first changes whether the target is flipped.)
- X on the target is unchanged: $I \otimes X \to I \otimes X$ (3) (the target flip commutes with "flip if control is 1").
- Z on the control is unchanged: $Z \otimes I \to Z \otimes I$ (4) (the control value is never changed).
- Z on the target spreads *back* to the control: $I \otimes Z \to Z \otimes Z$ (1): after the CNOT the target holds $z_0 \oplus z_1$.

The last rule is the heart of the cost gate: $\mathrm{CNOT}\cdot(I \otimes R_Z(\theta))\cdot\mathrm{CNOT} = e^{-i\theta\, Z\otimes Z/2}$.
Why the other options are incorrect.
(A) P→3, Q→2, R→1, S→4 reverses both propagation directions: it lets X move from target to control and Z from control to target (e.g. it changes $Z \otimes I$, although the CNOT never alters the control bit).
(B) R→1, S→4 lets Z spread from the control to the target; Z spreads the other way (target → control).
(C) P→3, Q→2 lets X spread from the target to the control; X spreads from control to target.
Key takeaway. X flows control → target, Z flows target → control; CNOT-R_Z-CNOT is a ZZ rotation.

Q.28 SINGLE-CORRECT MCQ · Entanglement by the cost gate

The state $e^{-i\gamma\, Z\otimes Z/2}\,|+\rangle\,|+\rangle$ is maximally entangled for which $\gamma \in (0,\pi)$?

(A) ✓ $\gamma = \pi/2$ **(B)** $\gamma = 3\pi/4$

(C) $\gamma = \pi/4$ **(D)** for no γ: it is always a product state

Answer: (A) Detailed solution

Correct answer. **(A)** $\gamma = \pi/2$

Concepts required. Product-state test and concurrence of a two-qubit pure state.

Relevant formulae. $|\psi\rangle = \sum c_{ab}\,|ab\rangle$ is a product state $\iff$ $c_{00}c_{11} = c_{01}c_{10}$; concurrence $\mathcal{C} = 2|c_{00}c_{11} - c_{01}c_{10}|$.

Step-by-step explanation. Amplitudes: $c_{00} = c_{11} = \frac{1}{2}e^{-i\gamma/2}$, $c_{01} = c_{10} = \frac{1}{2}e^{i\gamma/2}$. A two-qubit pure state is a product state iff $c_{00}c_{11} - c_{01}c_{10} = 0$; the concurrence is $\mathcal{C} = 2|c_{00}c_{11} - c_{01}c_{10}| = 2\cdot\frac{1}{4}|e^{-i\gamma} - e^{i\gamma}| = |\sin\gamma|$. $\mathcal{C} = 1$ (maximal) requires $\sin\gamma = \pm 1$, i.e. $\gamma = \pi/2$ in $(0,\pi)$; at $\gamma = \pi/4$ or $3\pi/4$, $\mathcal{C} = 0.71$. The cost layer is the only entangling part of QAOA; the mixer acts on single qubits.

Why the other options are incorrect.

(B) At $3\pi/4$, also $\mathcal{C} \approx 0.71$.

(C) At $\gamma = \pi/4$, $\mathcal{C} = \sin(\pi/4) \approx 0.71$: entangled, but not maximally.

(D) For $\gamma \notin \pi\mathbb{Z}$, $c_{00}c_{11} = \frac{1}{4}e^{-i\gamma} \neq \frac{1}{4}e^{i\gamma} = c_{01}c_{10}$: the state is entangled.

Key takeaway. Cost layers entangle (most strongly at $\gamma w = \pi/2$); mixers do not.

Q.29 STATEMENT I / II · Global phase vs. sign of γ

Statement I: Replacing H_C by $H_C + c\,I$ (any real constant c) changes the QAOA state $|\boldsymbol{\gamma},\boldsymbol{\beta}\rangle$ only by a global phase.

Statement II: Consequently, a QAOA circuit that uses $e^{-i\gamma C}$ with $C = \frac{W}{2} - H_C$ has the same optimal angles (γ^*,β^*) as the notebook's circuit with $e^{-i\gamma H_C}$.

(A) Both Statement I and Statement II are true

(B) Both Statement I and Statement II are false

(C) ✓ Statement I is true, but Statement II is false

(D) Statement I is false, but Statement II is true

Answer: (C) Detailed solution

Correct answer. **(C)** Statement I is true, but Statement II is false

Concepts required. Global phases are unobservable; the sign of the generator matters.

Relevant formulae. $e^{-i\gamma(H+cI)} = e^{-i\gamma c}e^{-i\gamma H}$; $C = \frac{W}{2}I - H_C \Rightarrow e^{-i\gamma C} \simeq e^{+i\gamma H_C}$.

Step-by-step explanation. **I is true:** cI commutes with everything, so $e^{-i\gamma(H_C+cI)} = e^{-i\gamma c}\,e^{-i\gamma H_C}$; the factors $e^{-i\gamma_l c}$ multiply the final state by an unobservable global phase. **II is false:** $C = \frac{W}{2}I - H_C$ contains $-H_C$, so $e^{-i\gamma C} = e^{-i\gamma W/2}\,e^{+i\gamma H_C}$. Apart from a global phase this is the notebook's cost unitary with $\gamma \to -\gamma$. The optimal angles are related by $\gamma^* \to -\gamma^*$ (the constant is harmless, the *sign* is not). This is why textbook formulas (written with $e^{-i\gamma C}$) must be translated before they are used with the notebook's convention. Answer: (C).

Why the other options are incorrect.

(A) needs both statements true; II is false: the cut operator contains $-H_C$, which reverses the sign of γ.

(B) needs both statements false; I is true.

(D) needs I false and II true; I is true and II is false.

Key takeaway. Constants are harmless, signs are not: $\gamma_{\text{textbook}} = -\gamma_{\text{notebook}}$.

Q.30 SINGLE-CORRECT MCQ · $\langle Z_0 Z_1 \rangle$ for a product state

Two qubits are in a product state in which $P(z_0 = 1) = 0.2$ and $P(z_1 = 1) = 0.7$. The value of $\langle Z_0 Z_1 \rangle$ is

(A) $+0.24$ **(B)** ✓ -0.24 **(C)** 0.14 **(D)** 0.62

Answer: (B) Detailed solution

Correct answer. (B) -0.24
Concepts required. Expectation values factorise for product states.
Relevant formulae. $\langle Z \rangle = P(0) - P(1)$; product state: $\langle Z_0 Z_1 \rangle = \langle Z_0 \rangle \langle Z_1 \rangle$; $\langle Z_0 Z_1 \rangle = 1 - 2P(z_0 \neq z_1)$.
Step-by-step explanation. For a product state the expectation factorises: $\langle Z_0 Z_1 \rangle = \langle Z_0 \rangle \langle Z_1 \rangle$, with $\langle Z \rangle = P(0) - P(1)$. So $\langle Z_0 \rangle = 0.8 - 0.2 = 0.6$ and $\langle Z_1 \rangle = 0.3 - 0.7 = -0.4$, giving -0.24. Check via the cut probability: $P(z_0 \neq z_1) = 0.2 \cdot 0.3 + 0.8 \cdot 0.7 = 0.62$ and $\langle Z_0 Z_1 \rangle = 1 - 2 \cdot 0.62 = -0.24$.
Why the other options are incorrect.
(A) $+0.24$ has the wrong sign: $\langle Z_1 \rangle = 0.3 - 0.7 < 0$.
(C) $0.14 = 0.2 \times 0.7$ is $P(z_0 = 1, z_1 = 1)$.
(D) 0.62 is $P(z_0 \neq z_1)$, the probability that the edge is cut, not $\langle Z_0 Z_1 \rangle$.
Key takeaway. Translate between $\langle Z_i Z_j \rangle$ and $P(\text{cut})$ with $\langle Z_i Z_j \rangle = 1 - 2P(\text{cut})$.

Q.31 SINGLE-CORRECT MCQ · Conjugation $e^{i\beta X} Z e^{-i\beta X}$

Conjugating Z by the mixer gate gives $e^{i\beta X}\, Z\, e^{-i\beta X} =$

(A) $\cos 2\beta\, Z - \sin 2\beta\, Y$ **(B)** ✓ $\cos 2\beta\, Z + \sin 2\beta\, Y$ **(C)** $\cos\beta\, Z + \sin\beta\, Y$ **(D)** $\cos 2\beta\, Z + \sin 2\beta\, X$

SILICO FELLER

Answer: (B) Detailed solution

Correct answer. (B) $\cos 2\beta\, Z + \sin 2\beta\, Y$
Concepts required. Heisenberg picture: conjugation of a Pauli by a rotation about another axis.
Relevant formulae. $e^{i\beta X} Z = Z e^{-i\beta X}$; $e^{-2i\beta X} = \cos 2\beta - i \sin 2\beta\, X$; $ZX = iY$.
Step-by-step explanation. Because $XZ = -ZX$, moving Z through $e^{i\beta X}$ flips the sign of the exponent: $e^{i\beta X} Z = Z\, e^{-i\beta X}$. Hence

$$e^{i\beta X} Z e^{-i\beta X} = Z\, e^{-2i\beta X} = Z(\cos 2\beta - i \sin 2\beta\, X) = \cos 2\beta\, Z - i \sin 2\beta\, ZX.$$

With $ZX = iY$: $-i \sin 2\beta\, (iY) = +\sin 2\beta\, Y$. Result $\cos 2\beta\, Z + \sin 2\beta\, Y$: a rotation of the Bloch vector by 2β about the x axis (the angle doubles, as for every $e^{-i\phi P}$). This identity is the first step of the $p = 1$ derivation.
Why the other options are incorrect.
(A) $-\sin 2\beta\, Y$ comes from using $ZX = -iY$; actually $ZX = +iY$.
(C) $\cos\beta\, Z + \sin\beta\, Y$ misses the doubling: conjugation by $e^{-i\beta X}$ rotates by 2β.
(D) X commutes with $e^{i\beta X}$; rotating Z about x produces a Y component, never an X component.
Key takeaway. Rotations by $e^{-i\phi P}$ turn the Bloch vector by 2ϕ.

Q.32 SINGLE-CORRECT MCQ · State-vector memory on one GPU

A state vector needs 16×2^n bytes in fp64 and 8×2^n bytes in fp32. Ignoring simulator workspace, the largest n whose state fits on a single 80 GB (80×10^9 bytes) GPU is

(A) 33 (fp64) and 34 (fp32)
(B) 36 (fp64) and 37 (fp32)
(C) ✓ 32 (fp64) and 33 (fp32)
(D) 32 (fp64) and 64 (fp32)

Answer: (C) Detailed solution

Correct answer. (C) 32 (fp64) and 33 (fp32)
Concepts required. State-vector memory grows as 2^n; precision sets bytes per amplitude.
Relevant formulae. fp64: 16×2^n bytes (complex128); fp32: 8×2^n bytes (complex64).
Step-by-step explanation. fp64: $16 \times 2^{32} = 6.87 \times 10^{10}$ bytes = 68.7 GB $\leq$ 80 GB, but $16 \times 2^{33} =$ 137.4 GB $>$ 80 GB, so $n = 32$. fp32 halves the memory: $8 \times 2^{33} = 68.7$ GB fits, $8 \times 2^{34} = 137.4$ GB does not, so $n = 33$. Halving the precision buys exactly one qubit. The notebook's 15-qubit state needs $16 \times 2^{15} = 524\,288$ bytes ("524.29 KB").
Why the other options are incorrect.
(A) $n = 33$ in fp64 needs 137 GB, more than 80 GB.
(B) 36 qubits need 1.1 TB in fp64.
(D) Halving the memory per amplitude adds one qubit, not 32.
Key takeaway. Every added qubit doubles the memory; fp32 buys exactly one qubit.

Q.33 SINGLE-CORRECT MCQ · Bit-flip probability of R_X

The gate $R_X(2\beta)$ with $\beta = \pi/8$ is applied to $|0\rangle$. The probability of measuring 1 is

(A) $\sin(\pi/8) \approx 0.383$
(B) $\frac{1}{2}$
(C) ✓ $\frac{2-\sqrt{2}}{4} \approx 0.146$
(D) $\frac{2+\sqrt{2}}{4} \approx 0.854$

Answer: (C) Detailed solution

Correct answer. (C) $\frac{2-\sqrt{2}}{4} \approx 0.146$
Concepts required. Action of R_X on $|0\rangle$; Born rule; half-angle identity.
Relevant formulae. $R_X(2\beta)|0\rangle = \cos\beta\,|0\rangle - i\sin\beta\,|1\rangle$; $\sin^2\phi = \frac{1-\cos 2\phi}{2}$.
Step-by-step explanation. $R_X(2\beta)|0\rangle = \cos\beta\,|0\rangle - i\sin\beta\,|1\rangle$ (first column of the matrix in Q.23), so $P(1) = \sin^2\beta$. For $\beta = \pi/8$: $\sin^2(\pi/8) = \frac{1-\cos(\pi/4)}{2} = \frac{1-\sqrt{2}/2}{2} = \frac{2-\sqrt{2}}{4} \approx 0.146$. $\sin(\pi/8)$ is the amplitude's modulus, not the probability.
Why the other options are incorrect.
(A) $0.383 = \sin(\pi/8)$ is the modulus of the amplitude, not its square.
(B) $\frac{1}{2}$ would need $\beta = \pi/4$.
(D) $0.854 = \cos^2(\pi/8)$ is $P(0)$.
Key takeaway. $R_X(2\beta)$ flips a bit with probability $\sin^2\beta$.

Q.34 Single-correct MCQ · $|+\rangle^{\otimes n}$ as top eigenstate of H_M

Which statement about $|+\rangle^{\otimes n}$ and $H_M = \sum_i X_i$ is correct?

(A) It is an eigenstate of H_M with eigenvalue 0

(B) ✓ It is the eigenstate of H_M with the *largest* eigenvalue, $+n$

(C) It is not an eigenstate of H_M

(D) It is the ground state of H_M, with eigenvalue $-n$

Answer: (B) Detailed solution

Correct answer. (B) It is the eigenstate of H_M with the *largest* eigenvalue, $+n$
Concepts required. Eigenstates of X and of sums of commuting single-qubit operators.
Relevant formulae. $X|\pm\rangle = \pm|\pm\rangle$; $H_M|+\rangle^{\otimes n} = n|+\rangle^{\otimes n}$; spectrum of H_M: $\{-n, -n+2, \ldots, n\}$.
Step-by-step explanation. $X|+\rangle = +|+\rangle$, so each X_i gives $+1$ on $|+\rangle^{\otimes n}$ and $H_M|+\rangle^{\otimes n} = n|+\rangle^{\otimes n}$. The eigenvalues of H_M range from $-n$ (state $|-\rangle^{\otimes n}$) to $+n$, so $|+\rangle^{\otimes n}$ is the *highest* eigenstate of H_M (equivalently the ground state of $-H_M$). This matters for the annealing interpretation: adiabatic evolution keeps a state in the *same* position of the spectrum, so with $+H_M$ the matching target is the highest state of the final Hamiltonian (see Q.54).
Why the other options are incorrect.
(A) Eigenvalue 0 (for even n) belongs to states with equally many $|+\rangle$ and $|-\rangle$ factors.
(C) $|+\rangle^{\otimes n}$ is an eigenstate: each X_i leaves it unchanged.
(D) The ground state of $\sum_i X_i$ is $|-\rangle^{\otimes n}$ with eigenvalue $-n$; $|+\rangle^{\otimes n}$ is the ground state of $-H_M$.
Key takeaway. The start state is the *top* state of $+H_M$ — remember this when reading QAOA as annealing.

Q.35 Assertion–Reason · Complex-conjugation symmetry

Assertion (A): A state $|\psi\rangle$ and its complex conjugate $|\psi^*\rangle$ (every amplitude conjugated) give identical computational-basis measurement statistics.

Reason (R): The initial QAOA state $|+\rangle^{\otimes n}$ has only real amplitudes.

(A) Both Assertion (A) and Reason (R) are true, and R is the correct explanation of A

(B) ✓ Both A and R are true, but R is *not* the correct explanation of A

(C) A is true, but R is false

(D) A is false, but R is true

Answer: (B) Detailed solution

Correct answer. (B) Both A and R are true, but R is *not* the correct explanation of A
Concepts required. Measurement probabilities depend only on moduli of amplitudes; complex conjugation symmetry of QAOA.
Relevant formulae. $|a^*|^2 = |a|^2$; real H_C, H_M, real start state $\Rightarrow |\boldsymbol{\gamma}, \boldsymbol{\beta}\rangle^* = |-\boldsymbol{\gamma}, -\boldsymbol{\beta}\rangle$.
Step-by-step explanation. **A is true:** the outcome probabilities are $|a_z|^2$ and $|a_z^*|^2 = |a_z|^2$ for every z, for *any* state. **R is true:** all amplitudes of $|+\rangle^{\otimes n}$ equal $2^{-n/2}$. But R does not explain A: A holds for every state, real or not; the explanation is $|a^*| = |a|$. (R *is* the reason for a different fact: since H_C, H_M and $|+\rangle^{\otimes n}$ are real, $|\boldsymbol{\gamma}, \boldsymbol{\beta}\rangle^* = |-\boldsymbol{\gamma}, -\boldsymbol{\beta}\rangle$, so $F_p(-\boldsymbol{\gamma}, -\boldsymbol{\beta}) = F_p(\boldsymbol{\gamma}, \boldsymbol{\beta})$.) Answer: (B).
Why the other options are incorrect.
(A) needs A true, R true *and* R explaining A; R does not explain A, which holds for every state.
(C) needs A true and R false; R is true.
(D) needs A false and R true; A is true.
Key takeaway. $F_p(-\boldsymbol{\gamma}, -\boldsymbol{\beta}) = F_p(\boldsymbol{\gamma}, \boldsymbol{\beta})$: the landscape is point-symmetric.

Q.36 Single-correct MCQ · Linearity of $\langle H_C \rangle$

For the triangle K_3 (unit weights) a state has $\langle Z_0Z_1 \rangle = -0.6$, $\langle Z_1Z_2 \rangle = -0.2$ and $\langle Z_0Z_2 \rangle = 0.4$. Its expected cut $\langle C \rangle$ is

(A) -0.2 **(B)** 1.9 **(C)** 1.3 **(D)** ✓ 1.7

Answer: (D) Detailed solution

Correct answer. **(D)** 1.7
Concepts required. Linearity of expectation values; $\langle C \rangle$ from Pauli expectations.
Relevant formulae. $\langle H_C \rangle = \sum \frac{w_{ij}}{2} \langle Z_iZ_j \rangle$; $\langle C \rangle = \frac{W}{2} - \langle H_C \rangle$.
Step-by-step explanation. By linearity $\langle H_C \rangle = \frac{1}{2}(-0.6 - 0.2 + 0.4) = -0.2$, and $\langle C \rangle = \frac{W}{2} - \langle H_C \rangle = 1.5 + 0.2 = 1.7$. Edge by edge: $\langle C_{ij} \rangle = \frac{1}{2}(1 - \langle Z_iZ_j \rangle)$ gives $0.8 + 0.6 + 0.3 = 1.7$. This is how `cudaq.observe` works: it evaluates $\langle H \rangle = \sum_k c_k \langle P_k \rangle$ term by term. The value is below $C_{\max} = 2$, as it must be.
Why the other options are incorrect.
(A) -0.2 is $\langle H_C \rangle$, not $\langle C \rangle$.
(B) $1.9 = 1.5 + 0.4$ forgets the factor $\frac{1}{2}$ in front of the Z_iZ_j terms.
(C) $1.3 = 1.5 - 0.2$ uses $+\langle H_C \rangle$ instead of $-\langle H_C \rangle$.
Key takeaway. Per edge: $\langle C_{ij} \rangle = \frac{1}{2}(1 - \langle Z_iZ_j \rangle)$, then add.

Q.37 Single-correct MCQ · Mixer at $\beta = \pi/2$; period $\pi/2$

On n qubits, $e^{-i(\pi/2)H_M}$ with $H_M = \sum_i X_i$ equals ..., and therefore (since $C(z) = C(\bar{z})$) F_p is periodic in every single β_l with period ...

(A) ✓ $(-i)^n X^{\otimes n}$; $\pi/2$ **(B)** $(-i)^n X^{\otimes n}$; π
(C) I; $\pi/2$ **(D)** $X^{\otimes n}$; 2π

Answer: (A) Detailed solution

Correct answer. **(A)** $(-i)^n X^{\otimes n}$; $\pi/2$
Concepts required. Mixer at special angles; global bit-flip symmetry of the cut.
Relevant formulae. $e^{-i\frac{\pi}{2}X} = -iX$; $X^{\otimes n} Z_iZ_j X^{\otimes n} = Z_iZ_j$; $C(\bar{z}) = C(z)$.
Step-by-step explanation. $e^{-i(\pi/2)X} = \cos\frac{\pi}{2} I - i \sin\frac{\pi}{2} X = -iX$, and the mixer factorises over qubits, so $e^{-i(\pi/2)H_M} = (-iX)^{\otimes n} = (-i)^n X^{\otimes n}$: a global phase times "flip every bit". Shifting $\beta_l \to \beta_l + \pi/2$ inserts $X^{\otimes n}$ after layer l. $X^{\otimes n}$ commutes with every Z_iZ_j (two sign flips) and with H_M, so it can be moved to the end, where it maps $|z\rangle \to |\bar{z}\rangle$; since $C(\bar{z}) = C(z)$ the expectation is unchanged. Hence period $\pi/2$ in each β_l — one quarter of the naive 2π.
Why the other options are incorrect.
(B) The operator is right, but the period is $\pi/2$: the flip it produces is already a symmetry.
(C) $e^{-i(\pi/2)H_M}$ is not the identity; it flips every bit.
(D) The global phase $(-i)^n$ is missing (harmless) and the period is wrong: 2π is only the naive period.
Key takeaway. Search β_l in an interval of length $\pi/2$ only.

Q.38 Statement I / II · Measuring Z_iZ_j from shots

Statement I: Z_iZ_j has eigenvalue -1 exactly on the computational basis states with $z_i \neq z_j$.

Statement II: $\langle Z_iZ_j \rangle$ can be estimated from the same computational-basis shots that are used to sample candidate splits.

(A) ✓ Both Statement I and Statement II are true

(B) Both Statement I and Statement II are false

(C) Statement I is true, but Statement II is false

(D) Statement I is false, but Statement II is true

Answer: (A) Detailed solution

Correct answer. **(A)** Both Statement I and Statement II are true

Concepts required. Diagonal observables are measured in the computational basis.

Relevant formulae. $Z_iZ_j\,|z\rangle = (-1)^{z_i \oplus z_j}\,|z\rangle$; $\widehat{\langle Z_iZ_j \rangle} = \frac{1}{S}\sum_{\text{shots}}(-1)^{z_i \oplus z_j}$.

Step-by-step explanation. **I is true:** $Z_iZ_j\,|z\rangle = s_is_j\,|z\rangle$ with $s = (-1)^z$, and $s_is_j = -1 \iff z_i \neq z_j$. **II is true:** Z_iZ_j is diagonal in the computational basis, so each shot z directly yields the value $(-1)^{z_i \oplus z_j}$; averaging over shots estimates $\langle Z_iZ_j \rangle$ (and $\langle H_C \rangle$) with no extra basis change. This is what happens on hardware targets, where the notebook estimates the objective from shots. Answer: (A).

Why the other options are incorrect.

(B) needs both statements false; both are true.

(C) needs I true and II false; II is true.

(D) needs I false and II true; I is true.

Key takeaway. One set of shots gives both the candidate splits and an estimate of the objective.

Paragraph for Questions 39 and 40

A single edge $(0, 1)$ with weight 1 is treated with one QAOA layer in the notebook's convention. Starting from $|00\rangle$ the gates are: H on both qubits; CNOT (control 0, target 1); $R_Z(\theta)$ on qubit 1 with $\theta = \gamma w = \gamma$; CNOT (control 0, target 1); $R_X(2\beta)$ on both qubits. The angles are $\gamma = \pi/2$ and $\beta = \pi/8$. Amplitudes are listed in the order $|00\rangle, |01\rangle, |10\rangle, |11\rangle$ (qubit 0 written first).

Q.39 Paragraph based · Trace: phase after the cost gate

Immediately after the second CNOT (before the mixer), the phase of the amplitude of $|01\rangle$ relative to the amplitude of $|00\rangle$ is

(A) 45° **(B)** 180° **(C)** ✓ 90° **(D)** 0°

Answer: (C) Detailed solution

Correct answer. **(C)** 90°

Concepts required. Cost gate as a diagonal phase: equal bits and different bits get opposite phases.

Relevant formulae. $e^{-i\gamma Z_0Z_1/2}\,|z_0z_1\rangle = e^{-i\frac{\gamma}{2}(-1)^{z_0 \oplus z_1}}\,|z_0z_1\rangle$.

Step-by-step explanation. After the Hadamards every amplitude is $\frac{1}{2}$. CNOT$\cdot(I \otimes R_Z(\gamma))\cdot$CNOT $= e^{-i\gamma Z_0Z_1/2}$: the first CNOT writes $z_0 \oplus z_1$ on qubit 1, $R_Z(\gamma)$ gives phase $e^{-i\gamma/2}$ if that bit is 0 and $e^{+i\gamma/2}$ if it is 1, and the second CNOT restores the bits. With $\gamma = \pi/2$: $|00\rangle, |11\rangle \to \frac{1}{2}e^{-i\pi/4}$ and $|01\rangle, |10\rangle \to \frac{1}{2}e^{+i\pi/4}$. Relative phase of $|01\rangle$ to $|00\rangle$: $e^{i\pi/4}/e^{-i\pi/4} = e^{i\pi/2}$, i.e. 90° $(= \gamma)$. The measurement probabilities are still all $\frac{1}{4}$.

Why the other options are incorrect.

(A) 45° is the phase of each amplitude, $\pm\gamma/2$; the *relative* phase is γ.

(B) 180° would need $\gamma = \pi$.

(D) 0° would mean the cost layer does nothing; $|00\rangle$ and $|01\rangle$ get opposite phases.

Key takeaway. The cost layer marks cut and uncut strings with a relative phase γw.

Q.40 PARAGRAPH BASED · Trace: cut probability after the mixer

After the mixer, the probability of measuring a *cut* bitstring (01 or 10) is

(A) 1 **(B)** $\frac{3}{4}$ **(C)** ✓ 0 **(D)** $\frac{1}{2}$

Answer: (C) Detailed solution

Correct answer. (C) 0

Concepts required. $p = 1$ single-edge formula; the sign relation between β and γ.

Relevant formulae. $\langle Z_0 Z_1 \rangle = \sin 4\beta \sin \gamma$, $P(\text{cut}) = \frac{1}{2}(1 - \sin 4\beta \sin \gamma)$.

Step-by-step explanation. For one edge with one layer in this convention $\langle Z_0 Z_1 \rangle = \sin 4\beta \sin \gamma$ (derived in the Answer Book of Q.47; the derivation uses Q.31 twice and the conjugation of $Y_0 Z_1$ by the cost gate). At $\gamma = \pi/2$, $\beta = \pi/8$: $\sin(\pi/2)\sin(\pi/2) = 1$, so $\langle Z_0 Z_1 \rangle = +1$ and $P(\text{cut}) = \frac{1}{2}(1 - \langle Z_0 Z_1 \rangle) = 0$. Explicitly, the final state has support only on $|00\rangle$ and $|11\rangle$ (state-vector simulation gives the probabilities $0.5, 0, 0, 0.5$ for $|00\rangle, |01\rangle, |10\rangle, |11\rangle$). With $\beta = -\pi/8$ instead, the probability of a cut bitstring is 1. The sign of β relative to γ decides whether the layer helps or hurts.

Why the other options are incorrect.

(A) 1 is the result for $\beta = -\pi/8$, not $+\pi/8$.

(B) $\frac{3}{4}$ is the ring's best value per edge at $p = 1$, unrelated here.

(D) $\frac{1}{2}$ would mean the layer did nothing (as at $\gamma = 0$ or $\beta = 0$).

Key takeaway. Same $|\beta|$, opposite sign: the layer can solve the edge perfectly or make it perfectly wrong.

SECTION C · QAOA Basics Q.41–60

Q.41 Single-correct MCQ · `cost_hamiltonian()` output

For `edges = [(0, 1, 2.0), (1, 2, 1.0)]` the notebook's function `cost_hamiltonian()` returns the operator

(A) ✓ $1.0\,Z_0Z_1 + 0.5\,Z_1Z_2$
(B) $-1.0\,Z_0Z_1 - 0.5\,Z_1Z_2$
(C) $2.0\,Z_0Z_1 + 1.0\,Z_1Z_2$
(D) $1.5\,I - 1.0\,Z_0Z_1 - 0.5\,Z_1Z_2$

Answer: (A) Detailed solution

Correct answer. **(A)** $1.0\,Z_0Z_1 + 0.5\,Z_1Z_2$
Concepts required. The notebook's cost Hamiltonian has coefficient $w/2$ per edge and no constant; the cut operator is $C = \frac{W}{2} - H_C$.
Relevant formulae. $H_C = \sum_{(i,j)\in E} \frac{w_{ij}}{2} Z_iZ_j$ (code: `0.5 * w * spin.z(i) * spin.z(j)`); $W = \sum w_{ij}$.
Step-by-step explanation. The code builds `term = 0.5 * w * cudaq.spin.z(i) * cudaq.spin.z(j)` for every edge and adds the terms: $H_C = \sum \frac{w_{ij}}{2} Z_iZ_j = \frac{2.0}{2} Z_0Z_1 + \frac{1.0}{2} Z_1Z_2 = 1.0\,Z_0Z_1 + 0.5\,Z_1Z_2$. No constant term is included; the cut is recovered as $C = \frac{W}{2} - H_C$ with $W = 3$, i.e. $C = 1.5\,I - H_C$ — the third option is the cut operator C, not what the function returns.
Why the other options are incorrect.
(B) The negative sign belongs to C, not to H_C; with it, minimising would find the *minimum* cut.
(C) $2.0\,Z_0Z_1 + 1.0\,Z_1Z_2$ omits the factor $\frac{1}{2}$ that comes from $[z_i \neq z_j] = \frac{1}{2}(1 - s_is_j)$.
(D) $1.5\,I - H_C$ is the cut operator C; the function returns H_C only.
Key takeaway. Read the code: `cost_hamiltonian()` $= \sum \frac{w}{2} Z_iZ_j$, and $C = W/2 - H_C$.

Q.42 Single-correct MCQ · All $\gamma_l = 0$

In the notebook's convention, if all cost angles are zero ($\gamma_1 = \cdots = \gamma_p = 0$), the expected cut $F_p(\boldsymbol{\gamma}, \boldsymbol{\beta})$ equals

(A) $W/2 + \frac{1}{2}\sum_l \sin 4\beta_l$
(B) $C_{\max}$
(C) ✓ $W/2$ for every choice of $\boldsymbol{\beta}$
(D) 0

Answer: (C) Detailed solution

Correct answer. **(C)** $W/2$ for every choice of $\boldsymbol{\beta}$
Concepts required. Without cost phases, the mixer acts on its own eigenstate.
Relevant formulae. $e^{-i\beta H_M}\,|+\rangle^{\otimes n} = e^{-i\beta n}\,|+\rangle^{\otimes n}$; $\langle C\rangle_{|+\rangle^{\otimes n}} = W/2$.
Step-by-step explanation. With $\gamma_l = 0$ every cost layer is the identity, so the state is $\prod_l e^{-i\beta_l H_M}\,|+\rangle^{\otimes n}$. $|+\rangle^{\otimes n}$ is an eigenstate of H_M with eigenvalue n (Q.34), so each mixer only multiplies it by the phase $e^{-i\beta_l n}$. The state remains $|+\rangle^{\otimes n}$ (up to a global phase), all 2^n splits are equally likely, and $\langle C\rangle = W/2$ (the random baseline; 15 for the notebook's graph). The cost layer creates the relative phases that the mixer turns into probability changes; without it nothing happens.
Why the other options are incorrect.
(A) There is no β dependence at all when every $\gamma_l = 0$; the extra term is invented.
(B) $C_{\max}$ would require the circuit to find the optimum without any information about the graph.
(D) 0 is the value of the strings $00\ldots0$ and $11\ldots1$, not of the uniform superposition.
Key takeaway. No cost phases, no progress: the mixer alone leaves the random baseline $W/2$.

Q.43 SINGLE-CORRECT MCQ · CNOT–R_Z–CNOT decomposition

In the notebook's kernel each edge (i, j) with weight w is implemented as `x.ctrl(q[i], q[j]); rz(gamma * w, q[j]); x.ctrl(q[i], q[j])`. With $R_Z(\theta) = e^{-i\theta Z/2}$, this sequence equals

(A) $e^{+i\gamma\,(w/2)\,Z_iZ_j}$ **(B)** $e^{-i\gamma\,w\,Z_iZ_j}$ **(C)** $e^{-i\gamma\,(w/2)\,Z_j}$ **(D)** ✓ $e^{-i\gamma\,(w/2)\,Z_iZ_j}$

Answer: (D) Detailed solution

Correct answer. (D) $e^{-i\gamma\,(w/2)\,Z_iZ_j}$

Concepts required. CNOT–R_Z–CNOT computes the parity $z_i \oplus z_j$ into a phase.

Relevant formulae. $R_Z(\theta)\,|b\rangle = e^{-i\theta(-1)^b/2}\,|b\rangle$; $\mathrm{CNOT}(I \otimes R_Z(\theta))\mathrm{CNOT} = e^{-i\theta Z\otimes Z/2}$.

Step-by-step explanation. Act on a basis state $|z_iz_j\rangle$. The first CNOT gives $|z_i,\, z_i \oplus z_j\rangle$. $R_Z(\theta)$ on qubit j multiplies by $e^{-i\theta/2}$ if $z_i \oplus z_j = 0$ and by $e^{+i\theta/2}$ if $z_i \oplus z_j = 1$, i.e. by $e^{-i(\theta/2)\,s_is_j}$ with $s = (-1)^z$. The second CNOT restores $|z_iz_j\rangle$. So the sequence is the diagonal operator $e^{-i(\theta/2)Z_iZ_j}$, and with $\theta = \gamma w$: $e^{-i\gamma(w/2)Z_iZ_j}$ — exactly the factor of $e^{-i\gamma H_C}$ for that edge. (Equivalently: CNOT conjugation maps $I \otimes Z$ to $Z \otimes Z$, see Q.27.)

Why the other options are incorrect.

(A) $R_Z(\theta)$ with $\theta = \gamma w > 0$ gives $e^{-i\cdots}$; the sign flips only for negative angles.

(B) $e^{-i\gamma w Z_iZ_j}$ forgets the half-angle convention of R_Z; it would correspond to `rz(2 * gamma * w)`.

(C) $e^{-i\gamma(w/2)Z_j}$ ignores the CNOTs, which turn the single-qubit Z_j into the parity Z_iZ_j.

Key takeaway. Two CNOTs and one $R_Z(\gamma w)$ per edge implement exactly $e^{-i\gamma\frac{w}{2}Z_iZ_j}$.

Q.44 SINGLE-CORRECT MCQ · Why `rx(2*beta)`

Why does the notebook apply `rx(2.0 * beta, q)` (and not `rx(beta, q)`) in the mixer layer?

(A) Because the mixer must be applied twice per layer

(B) Because the mixer Hamiltonian is $H_M = 2\sum_i X_i$

(C) Because β is measured in units of $\pi/2$

(D) ✓ Because $R_X(\theta) = e^{-i\theta X/2}$, so $R_X(2\beta) = e^{-i\beta X}$, the factor of $e^{-i\beta H_M}$ for one qubit

Answer: (D) Detailed solution

Correct answer. (D) Because $R_X(\theta) = e^{-i\theta X/2}$, so $R_X(2\beta) = e^{-i\beta X}$, the factor of $e^{-i\beta H_M}$ for one qubit

Concepts required. Half-angle convention of rotation gates.

Relevant formulae. $R_X(\theta) = e^{-i\theta X/2} \Rightarrow R_X(2\beta) = e^{-i\beta X}$; $e^{-i\beta H_M} = \prod_i e^{-i\beta X_i}$.

Step-by-step explanation. The target is $U_M(\beta) = e^{-i\beta H_M}$ with $H_M = \sum_i X_i$. Since the X_i commute, $e^{-i\beta H_M} = \prod_i e^{-i\beta X_i}$. CUDA-Q's `rx`, like the standard definition, is $R_X(\theta) = e^{-i\theta X/2}$ (half-angle convention). Matching $e^{-i\theta X/2} = e^{-i\beta X}$ gives $\theta = 2\beta$. The same half-angle convention is why the cost gate uses `rz(gamma * w)` for the coefficient $w/2$ (previous question).

Why the other options are incorrect.

(A) Each qubit receives exactly one `rx` per layer.

(B) $H_M = \sum_i X_i$ (code docstring and Section 1); the 2 comes from the gate convention, not from H_M.

(C) β is in radians; the period $\pi/2$ (Q.37) is a property of F_p, not a unit.

Key takeaway. Factor 2 in `rx(2*beta)`, factor w (not $w/2$) in `rz(gamma*w)`: both from $e^{-i\theta P/2}$.

Q.45 SINGLE-CORRECT MCQ · Gate and CNOT counts

Using the notebook's gate structure (H on every qubit, then per layer 3 gates per edge and one R_X per qubit), a graph with $n = 20$ nodes and $|E| = 57$ edges at depth $p = 3$ needs how many gates (excluding measurements), and how many of them are CNOTs?

(A) 573 and 342 **(B)** ✓ 593 and 342 **(C)** 593 and 171 **(D)** 402 and 228

Answer: (B) Detailed solution

Correct answer. (B) 593 and 342
Concepts required. Gate count of the notebook's QAOA circuit.
Relevant formulae. gates $= n + p\,(3|E| + n)$; CNOTs $= 2p|E|$; $R_Z = p|E|$; $R_X = pn$.
Step-by-step explanation. The notebook's estimate is `gates = N + p * (3 * len(edges) + N)`. Total: $n + p(3|E| + n) = 20 + 3\,(171 + 20) = 20 + 573 = 593$. CNOTs: 2 per edge per layer, $2p|E| = 2 \cdot 3 \cdot 57 = 342$. (The remaining $593 - 342 = 251$ are 20 H, 171 R_Z and 60 R_X.) For the default ($n = 15$, $|E| = 30$, $p = 2$) the same formula gives 225 gates and 120 CNOTs, as in the recorded statistics.
Why the other options are incorrect.
(A) 573 omits the initial layer of $n = 20$ Hadamards.
(C) 171 counts one CNOT per edge per layer; each edge needs two.
(D) 402 and 228 are the counts for $p = 2$, not $p = 3$.
Key takeaway. Two-qubit gates grow as $2p|E|$ — the dominant cost on hardware.

Q.46 MATCHING LISTS · Default circuit statistics

For the notebook's default run ($n = 15$ nodes, $|E| = 30$ edges, $p = 2$), match each quantity (List-I) with its value (List-II).

List-I		List-II	
(P)	number of variational parameters	**(1)**	105
(Q)	number of R_Z gates	**(2)**	120
(R)	number of single-qubit gates (H, R_Z and R_X together)	**(3)**	4
(S)	number of CNOT (`cx`) gates	**(4)**	60

(A) P→4, Q→3, R→1, S→2 **(B)** P→3, Q→4, R→2, S→1
(C) ✓ P→3, Q→4, R→1, S→2 **(D)** P→3, Q→2, R→1, S→4

Answer: (C) Detailed solution

Correct answer. (C) P→3, Q→4, R→1, S→2
Concepts required. Counting parameters and gates of the default circuit.
Relevant formulae. parameters $2p$; R_Z: $p|E|$; single-qubit: $n + p|E| + pn$; CNOT: $2p|E|$.
Step-by-step explanation.

- P: $2p = 4$ angles $(\gamma_1, \gamma_2, \beta_1, \beta_2)$ (3).
- Q: one R_Z per edge per layer: $p|E| = 60$ (4).
- R: n Hadamards $+p|E|$ R_Z $+pn$ R_X $= 15 + 60 + 30 = 105$ (1).
- S: two CNOTs per edge per layer: $2p|E| = 120$ (2).

These agree with the recorded `gate counts by type: h 15, cx 120, rz 60, rx 30` (225 gates

in total).

Why the other options are incorrect.

(A) P→4 would mean 60 parameters; the circuit has only $2p = 4$ angles regardless of the graph size.

(B) R→2, S→1 calls the 120 CNOTs single-qubit gates.

(D) Q→2, S→4 swaps R_Z and CNOT counts: there are two CNOTs but only one R_Z per edge per layer.

Key takeaway. Parameters: $2p$ (tiny); gates: proportional to $p(|E| + n)$.

Q.47 SINGLE-CORRECT MCQ · $p = 1$ single edge: formula, optimum

For a single unit edge and $p = 1$ in the notebook's convention, the expected cut is $F_1(\gamma, \beta) = \frac{1}{2} - \frac{1}{2}\sin 4\beta \sin\gamma$. Its maximum value, and one point where it is reached, are

(A) $\frac{3}{4}$, at $(\gamma, \beta) = (\pi/2, -\pi/8)$

(B) $\frac{1}{2}$, reached for all angles

(C) 1, at $(\gamma, \beta) = (\pi/2, \pi/8)$

(D) ✓ 1, at $(\gamma, \beta) = (\pi/2, -\pi/8)$

Answer: (D) Detailed solution

Correct answer. (D) 1, at $(\gamma, \beta) = (\pi/2, -\pi/8)$

Concepts required. Heisenberg-picture derivation of $\langle Z_0 Z_1\rangle$ for one edge and $p = 1$; sign convention.

Relevant formulae. $U_M^\dagger Z U_M = \cos 2\beta\, Z + \sin 2\beta\, Y$; $U_C^\dagger Y_0 Z_1 U_C = \cos\gamma\, Y_0 Z_1 + \sin\gamma\, X_0$; $F_1 = \frac{1}{2}(1 - \sin 4\beta \sin\gamma)$.

Step-by-step explanation. **Derivation of the formula.** $\langle Z_0 Z_1\rangle = \langle{+}{+}| U_C^\dagger U_M^\dagger Z_0 Z_1 U_M U_C |{+}{+}\rangle$. Mixer (Q.31 on each qubit, $c = \cos 2\beta$, $s = \sin 2\beta$): $U_M^\dagger Z_0 Z_1 U_M = (cZ_0 + sY_0)(cZ_1 + sY_1)$. Cost $U_C = e^{-i\gamma Z_0 Z_1/2}$: Z_0Z_1 and Y_0Y_1 commute with it; Y_0Z_1 anticommutes, so $U_C^\dagger Y_0 Z_1 U_C = Y_0 Z_1 e^{-i\gamma Z_0 Z_1} = \cos\gamma\, Y_0 Z_1 - i\sin\gamma\, Y_0 Z_0 = \cos\gamma\, Y_0 Z_1 + \sin\gamma\, X_0$ (using $YZ = iX$); likewise $Z_0 Y_1 \to \cos\gamma\, Z_0 Y_1 + \sin\gamma\, X_1$. In $|{+}{+}\rangle$ only X_0, X_1 have non-zero expectation (= 1), so $\langle Z_0 Z_1\rangle = 2cs\sin\gamma = \sin 4\beta \sin\gamma$ and $F_1 = \frac{1}{2}(1 - \sin 4\beta \sin\gamma)$. **Maximum.** $F_1 = 1$ needs $\sin 4\beta \sin\gamma = -1$: e.g. $\gamma = \pi/2$, $\beta = -\pi/8$ (or $\gamma = -\pi/2$, $\beta = \pi/8$). Then the edge is cut with certainty: for one edge, $p = 1$ already solves Max-Cut.

Why the other options are incorrect.

(A) At $(\pi/2, -\pi/8)$ the value is 1, not $\frac{3}{4}$; $\frac{3}{4}$ is the ring value (Q.48).

(B) F_1 does depend on the angles; it ranges over $[0, 1]$.

(C) $(\pi/2, \pi/8)$ gives $\sin 4\beta \sin\gamma = +1$ and $F_1 = 0$: the edge is *never* cut (Q.40). Textbooks using $e^{-i\gamma C}$ quote this point, but the notebook's convention flips the sign of γ.

Key takeaway. In the notebook's convention γ and β need opposite signs to increase the cut.

Q.48 Single-correct MCQ · $p = 1$ on the ring

With optimally chosen angles, the best approximation ratio of $p = 1$ QAOA on the ring (cycle) C_8 is

(A) 1 **(B)** ✓ 3/4 **(C)** 5/6 **(D)** 0.6924

Answer: (B) Detailed solution

Correct answer. **(B)** 3/4
Concepts required. $p = 1$ formula for triangle-free regular graphs; symmetry between the edges of a ring.
Relevant formulae. $\langle C_{uv}\rangle = \frac{1}{2} - \frac{1}{4}\sin 4\beta\,\sin\gamma\,(\cos^{d_u-1}\gamma + \cos^{d_v-1}\gamma)$; ring: $d = 2$.
Step-by-step explanation. The ring is 2-regular and triangle-free. Each edge has $d_u = d_v = 2$, so (formula printed in the passage for Q.59 and Q.60, which holds for every triangle-free graph) $\langle Z_u Z_v\rangle = \sin 4\beta \sin\gamma\cos\gamma = \frac{1}{2}\sin 4\beta\,\sin 2\gamma$, and per edge

$$\langle C_{uv}\rangle = \tfrac{1}{2} - \tfrac{1}{4}\sin 4\beta\,\sin 2\gamma \le \tfrac{1}{2} + \tfrac{1}{4} = \tfrac{3}{4},$$

reached at $\sin 4\beta \sin 2\gamma = -1$, e.g. $\gamma = \pi/4$, $\beta = -\pi/8$. All 8 edges are equivalent: $F_1 = 8 \cdot \frac{3}{4} = 6$ and $C_{\max} = 8$ (even ring, bipartite): $r = 6/8 = 3/4$. A numerical optimisation over the angles gives the same value.
Why the other options are incorrect.
(A) 1 is impossible at $p = 1$: $\frac{1}{4}|\sin 4\beta \sin 2\gamma| \le \frac{1}{4}$ caps each edge at $\frac{3}{4}$.
(C) 5/6 is the ring value at $p = 2$ (Q.62).
(D) 0.6924 is the 3-regular worst-case bound, not the ring value.
Key takeaway. Rings: $r_p = \frac{2p+1}{2p+2}$ for $p = 1, 2, 3$: $\frac{3}{4}, \frac{5}{6}, \frac{7}{8}$.

Q.49 Statement I / II · 0.6924 bound; monotonicity in p

Statement I: For $p = 1$ with optimal angles, QAOA achieves $\langle C\rangle \ge 0.6924\, C_{\max}$ on every 3-regular graph.

Statement II: For every graph, the optimal value $F_p^* = \max_{\gamma,\beta} F_p$ satisfies $F_{p+1}^* \ge F_p^*$.

(A) ✓ Both Statement I and Statement II are true
(B) Both Statement I and Statement II are false
(C) Statement I is true, but Statement II is false
(D) Statement I is false, but Statement II is true

Answer: (A) Detailed solution

Correct answer. **(A)** Both Statement I and Statement II are true
Concepts required. Known performance guarantee at $p = 1$; nesting of QAOA circuits of different depth.
Relevant formulae. $F_1^* \ge 0.6924\, C_{\max}$ (3-regular); $F_{p+1}(\boldsymbol{\gamma}, 0, \boldsymbol{\beta}, 0) = F_p(\boldsymbol{\gamma}, \boldsymbol{\beta}) \Rightarrow F_{p+1}^* \ge F_p^*$.
Step-by-step explanation. **I is true:** Farhi, Goldstone and Gutmann (2014) showed that $p = 1$ QAOA gives at least $0.6924\, C_{\max}$ on 3-regular graphs (worst case over local neighbourhoods). **II is true:** the depth-$(p+1)$ circuit contains the depth-p circuit as a special case: setting $\gamma_{p+1} = \beta_{p+1} = 0$ makes the extra layer the identity. So the maximum over a larger set cannot be smaller. (With a *finite* optimizer budget the *found* value can still be worse — more parameters are harder to optimise.) Answer: (A).
Why the other options are incorrect.
(B) needs both statements false; both are true.
(C) needs I true and II false; II is true.

(D) needs I false and II true; I is true.

Key takeaway. Optimal values never decrease with depth; found values can, if the optimizer struggles.

Q.50 SINGLE-CORRECT MCQ · Limit $p \to \infty$

Which statement about the limit of large depth p (angles optimised for each p) is correct?

(A) $F_p^*/C_{\max} \to 0.878$, the Goemans–Williamson ratio

(B) $F_p^* = C_{\max}$ already at $p = 1$ for every graph

(C) $F_p^* \to W/2$, because deep circuits randomise the state

(D) ✓ $F_p^* \to C_{\max}$ as $p \to \infty$, because a Trotterised adiabatic path is one admissible choice of angles

Answer: (D) Detailed solution

Correct answer. (D) $F_p^* \to C_{\max}$ as $p \to \infty$, because a Trotterised adiabatic path is one admissible choice of angles

Concepts required. Adiabatic theorem and Trotterisation as a special QAOA schedule.

Relevant formulae. $e^{-iT\int H(s)ds} \approx \prod_{l=1}^{p} e^{-i\beta_l H_M} e^{-i\gamma_l H_C}$ with small steps; $\lim_{p\to\infty} F_p^* = C_{\max}$.

Step-by-step explanation. Farhi et al. argue: a slow interpolation from the mixer to the cost Hamiltonian (adiabatic theorem), discretised into small time steps (Trotterisation), is a QAOA circuit with special angles. As $p \to \infty$ these special angles drive the initial state to an optimal split, so the *optimal* angles do at least as well: $F_p^* \to C_{\max}$. 0.878 is a classical polynomial-time guarantee, not a QAOA limit; $p = 1$ is exact only for special graphs (one edge, the triangle).

Why the other options are incorrect.

(A) 0.878 is the classical Goemans–Williamson guarantee; it is not a limit of QAOA.

(B) $p = 1$ is exact only for special graphs (one edge, triangle); e.g. rings reach only $\frac{3}{4}$ (Q.48).

(C) Deep circuits with *optimised* angles do not randomise the state; random angles would.

Key takeaway. QAOA is exact in the infinite-depth limit, approximate at any fixed depth.

Q.51 SINGLE-CORRECT MCQ · Parameter-vector layout

In the notebook the angle vector is `params` $= [\gamma_1, \dots, \gamma_p, \beta_1, \dots, \beta_p]$; the kernel reads `gamma = params[layer]` and `beta = params[layers + layer]`. For $p = 3$ and `params = [a, b, c, d, e, f]`, the mixer angle β_2 of the second layer is

(A) d **(B)** b **(C)** ✓ e **(D)** f

Answer: (C) Detailed solution

Correct answer. (C) e

Concepts required. Blocked parameter layout and zero-based layer index in the kernel.

Relevant formulae. $\gamma_{l+1} =$ `params[l]`, $\beta_{l+1} =$ `params[p + l]`, $l = 0, \dots, p-1$.

Step-by-step explanation. Layers are numbered `layer = 0, 1, 2` in the code. The second layer is `layer = 1`: `gamma = params[1] = b` and `beta = params[3 + 1] = params[4] = e`. So $\beta_2 = e$ (and $\gamma_2 = b$). The vector is *blocked* (all γ first, then all β), not interleaved ($\gamma_1, \beta_1, \gamma_2, \beta_2, \dots$).

Why the other options are incorrect.

(A) d would be right for an interleaved layout ($\gamma_1, \beta_1, \gamma_2, \beta_2, \dots$), which the notebook does not use.

(B) b is γ_2, the cost angle of the second layer.

(D) f is β_3, the mixer angle of the third layer.

Key takeaway. `params` = all γ's, then all β's.

Q.52 Single-correct MCQ · The ramp initialisation

With `INITIAL_PARAMETERS = "ramp"` the notebook starts from `np.linspace(0.2, 0.6, p)` for the γ's and `np.linspace(0.6, 0.2, p)` for the β's. For $p = 5$, the initial values of γ_3 and β_2 are

(A) 0.3 and 0.5 **(B)** 0.36 and 0.52 **(C)** 0.4 and 0.3 **(D)** ✓ 0.4 and 0.5

Answer: (D) Detailed solution

Correct answer. **(D)** 0.4 and 0.5
Concepts required. `np.linspace` includes both end points.
Relevant formulae. `linspace(a, b, p)`$_k = a + k\frac{b-a}{p-1}$, $k = 0, \ldots, p-1$.
Step-by-step explanation. `np.linspace(a, b, p)` returns p equally spaced values *including both end points*; the step is $(b-a)/(p-1)$. γ: step $(0.6-0.2)/4 = 0.1$: $(0.2, 0.3, 0.4, 0.5, 0.6)$, so $\gamma_3 = 0.4$. β: $(0.6, 0.5, 0.4, 0.3, 0.2)$, so $\beta_2 = 0.5$. (A step of $0.4/5 = 0.08$, which excludes an end point, would give 0.36 and 0.52.)
Why the other options are incorrect.
(A) 0.3 is γ_2, not γ_3.
(B) 0.36 and 0.52 use a step of $(b-a)/p$, which would not end exactly at b.
(C) 0.3 is β_4; β decreases from 0.6, so $\beta_2 = 0.5$.
Key takeaway. The ramp has rising γ and falling β with step $0.4/(p-1)$.

Q.53 Assertion–Reason · Minimise $\langle H_C \rangle$ = maximise $\langle C \rangle$

Assertion (A): The notebook's objective function returns $\langle H_C \rangle$ and SciPy *minimises* it; this is equivalent to *maximising* the expected cut.
Reason (R): $\langle C \rangle = W/2 - \langle H_C \rangle$, and $W/2$ does not depend on the angles.

(A) ✓ Both Assertion (A) and Reason (R) are true, and R is the correct explanation of A
(B) Both A and R are true, but R is *not* the correct explanation of A
(C) A is true, but R is false
(D) A is false, but R is true

Answer: (A) Detailed solution

Correct answer. **(A)** Both Assertion (A) and Reason (R) are true, and R is the correct explanation of A
Concepts required. Minimising $\langle H_C \rangle \Leftrightarrow$ maximising $\langle C \rangle$.
Relevant formulae. $\langle C \rangle = \frac{W}{2} - \langle H_C \rangle$ (code: `history_cut.append(total_weight / 2 - energy)`).
Step-by-step explanation. **R is true:** $[z_i \neq z_j] = \frac{1}{2}(1 - s_i s_j)$, so $C = \frac{W}{2} I - H_C$ as operators, and taking expectations gives R. **A is true because of R:** $\langle C \rangle$ is a constant minus $\langle H_C \rangle$, so the angles that minimise $\langle H_C \rangle$ are exactly those that maximise $\langle C \rangle$. The code records `history_cut.append(total_weight / 2 - energy)` after each evaluation and returns `energy` to `minimize`. R is the correct explanation. Answer: (A).
Why the other options are incorrect.
(B) needs A and R both true but R not explaining A; R is precisely the reason.
(C) needs A true and R false; R is true.
(D) needs A false and R true; A is true.
Key takeaway. A constant shift and a sign turn energy minimisation into cut maximisation.

Q.54 SINGLE-CORRECT MCQ · Sign of the default start point

At the notebook's default starting point $\boldsymbol{\gamma} = (0.2, 0.6)$, $\boldsymbol{\beta} = (0.6, 0.2)$, an exact state-vector calculation for its 15-node graph gives $\langle C \rangle \approx 9.18$, *below* the random baseline $W/2 = 15$. The best explanation is

(A) ✓ In the convention $e^{-i\beta H_M} e^{-i\gamma H_C}$, the start state $|+\rangle^{\otimes n}$ is the *highest* eigenstate of H_M, so an annealing-like ramp with $\gamma, \beta > 0$ heads for the highest eigenstate of H_C, the *smallest* cut; heading for the largest cut needs γ and β of opposite sign

(B) The value is meaningless: `cudaq.observe` returns random numbers before the optimizer has converged

(C) Every $p = 2$ starting point gives $\langle C \rangle < W/2$; only $p \geq 3$ can beat random guessing

(D) It indicates a bug, because a QAOA state can never have $\langle C \rangle < W/2$

Answer: (A) **Detailed solution**

Correct answer. **(A)** In the convention $e^{-i\beta H_M} e^{-i\gamma H_C}$, the start state $|+\rangle^{\otimes n}$ is the *highest* eigenstate of H_M, so an annealing-like ramp with $\gamma, \beta > 0$ heads for the highest eigenstate of H_C, the *smallest* cut; heading for the largest cut needs γ and β of opposite sign

Concepts required. Sign conventions; adiabatic interpretation; the start state is the top eigenstate of H_M.

Relevant formulae. Small angles, one edge: $\Delta\langle C \rangle \approx -2\beta\gamma$ (from $\frac{1}{2} - \frac{1}{2}\sin 4\beta \sin\gamma$); adiabatic evolution preserves the position in the spectrum.

Step-by-step explanation. **Small-angle check.** For one edge $F_1 = \frac{1}{2} - \frac{1}{2}\sin 4\beta \sin\gamma$ (Q.47): with $\gamma > 0$ and small $\beta > 0$ the cut *decreases*; it increases only if $\sin 4\beta \sin\gamma < 0$. **Annealing picture.** $e^{-i\beta H_M} e^{-i\gamma H_C}$ with positive angles is one Trotter step of evolution under $\beta H_M + \gamma H_C$. $|+\rangle^{\otimes n}$ is the *top* eigenstate of H_M (Q.34); an adiabatic ramp keeps the top eigenstate, which at the end is the top eigenstate of H_C — the *minimum* cut. For the maximum cut one needs $H_M \to -H_C$, i.e. γ and β of opposite sign. **Not a bug.** $\langle C \rangle$ can be anywhere between $C_{\min}$ and $C_{\max}$; the optimizer (COBYLA) moves away from the poor start, and the recorded run still ended at $\langle C \rangle = 20.53$ ($r = 0.82$). The value 9.18 is computed from the notebook's own conventions (`verify_answers.py` reproduces it with NumPy and CUDA-Q).

Why the other options are incorrect.

(B) `cudaq.observe` is deterministic on a state-vector simulator: it returns the exact $\langle H_C \rangle$ at the given angles.

(C) Many $p = 2$ angles beat $W/2$ — the optimizer reached 20.53 with $p = 2$.

(D) QAOA states can have any $\langle C \rangle$ between $C_{\min}$ and $C_{\max}$; values below $W/2$ are allowed.

Key takeaway. Check the sign of the start: with $e^{-i\gamma H_C}$ choose $\gamma\beta < 0$ to head for large cuts.

Q.55 MATCHING LISTS · Algorithm ↔ code

Match each part of the QAOA algorithm (List-I) with the code that implements it in the notebook (List-II).

List-I		List-II	
(P)	initial state $\lvert+\rangle^{\otimes n}$	**(1)**	`rx(2.0 * beta, qubits[q])`
(Q)	cost unitary for one edge	**(2)**	`cudaq.observe(kernels["state"], H_C, ...).expectation()`
(R)	mixer unitary for one qubit	**(3)**	`h(qubits)`
(S)	objective $\langle H_C \rangle$	**(4)**	`x.ctrl(...); rz(gamma * weights[e], ...); x.ctrl(...)`

(A) P→3, Q→1, R→4, S→2
(B) P→3, Q→4, R→2, S→1
(C) ✓ P→3, Q→4, R→1, S→2
(D) P→4, Q→3, R→1, S→2

Answer: (C) Detailed solution

Correct answer. **(C)** P→3, Q→4, R→1, S→2
Concepts required. Mapping the algorithm's building blocks to CUDA-Q calls.
Relevant formulae. $\lvert+\rangle^{\otimes n} = H^{\otimes n}\lvert 0\rangle^{\otimes n}$; $e^{-i\gamma\frac{w}{2}Z_iZ_j} = \text{CNOT } R_Z(\gamma w) \text{ CNOT}$; $e^{-i\beta X} = R_X(2\beta)$.
Step-by-step explanation.

- P: `h(qubits)` applies H to every qubit of the freshly allocated $\lvert 0\rangle^{\otimes n}$, giving $\lvert+\rangle^{\otimes n}$ (3).
- Q: CNOT, $R_Z(\gamma w)$, CNOT $= e^{-i\gamma(w/2)Z_iZ_j}$ (4).
- R: $R_X(2\beta) = e^{-i\beta X}$ (1).
- S: `cudaq.observe` computes $\langle\psi\lvert H_C\rvert\psi\rangle$ for the state-preparation kernel (exactly on simulators, from shots when `shots_count` is given) (2).

Why the other options are incorrect.
(A) Q→1, R→4 swaps cost and mixer: the mixer contains no two-qubit gates.
(B) R→2, S→1: `cudaq.observe` computes an expectation value; it does not apply gates to the circuit.
(D) P→4, Q→3: the Hadamard layer is the initial state; it is not the cost layer.
Key takeaway. `h` → start, `x.ctrl/rz/x.ctrl` → cost, `rx` → mixer, `observe` → objective.

Q.56 SINGLE-CORRECT MCQ · Triangle solved at $p = 1$

For the triangle K_3 (unit weights, $C_{\max} = 2$), the best value of F_1 over all angles is

(A) $\sqrt{3}$, i.e. $r \approx 0.866$
(B) ✓ 2, i.e. $r = 1$
(C) 1.75, i.e. $r = 0.875$
(D) 1.5, i.e. $r = 0.75$

Answer: (B) Detailed solution

Correct answer. **(B)** 2, i.e. $r = 1$
Concepts required. When H_C has only two eigenvalues, one layer can rotate the state fully into the optimal subspace.
Relevant formulae. $H_C = \frac{3}{2}\Pi_{\{000,111\}} - \frac{1}{2}\Pi_{\text{opt}}$; optimum $(\gamma^*, \beta^*) = (\arctan\frac{1}{\sqrt{2}}, -\frac{1}{2}\arctan\frac{1}{\sqrt{2}})$, $F_1 = 2$.
Step-by-step explanation. H_C has only two eigenvalues (Q.9): $+\frac{3}{2}$ on $\{000, 111\}$ and $-\frac{1}{2}$ on the six optimal strings. Hence $e^{-i\gamma H_C}$ only puts a relative phase $e^{-2i\gamma}$ between the two

non-optimal strings and the six optimal ones, and the mixer can then interfere the two non-optimal amplitudes away completely. A numerical maximisation gives $F_1 = 2$ exactly, for example at $\gamma^* = \arctan(1/\sqrt{2}) \approx 0.6155$, $\beta^* = -\gamma^*/2 \approx -0.3077$ (notebook convention; checked in `verify_answers.py`). So $p = 1$ already finds the optimum with probability 1. The value $1.5 = W/2$ is only the random baseline.

Why the other options are incorrect.

(A) $\sqrt{3}$ is not attained as a maximum; numerical optimisation reaches the full 2.

(C) 1.75 is not the maximum either; the triangle is solved exactly at $p = 1$.

(D) $1.5 = W/2$ is the random baseline (all angles zero).

Key takeaway. Small, highly symmetric instances can be solved exactly at $p = 1$; rings cannot.

Q.57 SINGLE-CORRECT MCQ · r vs. best sample

The previously recorded run reports: approximation ratio $r = \langle C \rangle / C_{\max} = 0.8213$, best sampled cut 25 ($= C_{\max}$), and $P(\text{optimal split}) = 0.06$ with 4000 shots. Which interpretation is correct?

(A) 82 % of the shots returned an optimal split

(B) ✓ r measures the *average* cut of the distribution; the best of 4000 samples can still be optimal, and here 240 shots were optimal

(C) The best sampled cut can never exceed $\langle C \rangle = 20.53$, so the value 25 indicates an error

(D) The best sampled split has cut $0.8213 \times 25 \approx 20.5$

Answer: (B) Detailed solution

Correct answer. (B) r measures the *average* cut of the distribution; the best of 4000 samples can still be optimal, and here 240 shots were optimal

Concepts required. Expectation versus best sample; probability of the optimum; QAOA as estimator and as sampler.

Relevant formulae. $r = \langle C \rangle / C_{\max}$; $\mathbb{E}[\#\text{optimal shots}] = P_{\text{opt}}\, S = 0.06 \times 4000 = 240$.

Step-by-step explanation. $\langle C \rangle = \sum_z |\langle z|\boldsymbol{\gamma}, \boldsymbol{\beta}\rangle|^2 C(z) = 20.53$ is an average over the whole output distribution; $r = 20.53/25 = 0.82$ summarises that average. Individual samples scatter around it: some are better than average. Among 4000 shots, $P_{\text{opt}} = 0.06$ means about $0.06 \times 4000 = 240$ optimal shots (exactly 240 were recorded), so the best sample reaches $C_{\max}$. QAOA used as a *sampler* is judged by the best sample (and P_{opt}), used as an *estimator* by r.

Why the other options are incorrect.

(A) 82 % of shots optimal would mean $P_{\text{opt}} = 0.82$; the recorded P_{opt} is 0.06.

(C) Individual samples scatter around the mean; the maximum of 4000 samples is typically far above the mean.

(D) 0.8213×25 is the *expected* cut 20.53, not the best sample.

Key takeaway. Report both: r (average quality) and the best sample with P_{opt} (what a user actually gets).

Q.58 STATEMENT I / II · Best evaluated angles; no global guarantee

Statement I: The notebook's final sampling run uses the angles of the best evaluation stored in `history_cut`, which need not be the optimizer's final point `opt.x`.

Statement II: With COBYLA and a budget of 80 evaluations, the notebook is guaranteed to find the global maximum of F_2.

(A) Both Statement I and Statement II are true

(B) Both Statement I and Statement II are false

(C) ✓ Statement I is true, but Statement II is false

(D) Statement I is false, but Statement II is true

Answer: (C) Detailed solution

Correct answer. **(C)** Statement I is true, but Statement II is false

Concepts required. The notebook's choice of final angles; local optimizers give no global guarantee.

Relevant formulae. `best_index = argmax(history_cut);` `best_params = history_params[best_index]`.

Step-by-step explanation. **I is true:** the code computes `best_index = int(np.argmax(history_cut))` and samples with `best_params = history_params[best_index]`; this protects against an optimizer that ends on a worse point. **II is false:** COBYLA is a local, derivative-free method; QAOA landscapes have many local optima, and the recorded run stopped with "*the objective function has been evaluated MAXFUN times*" — i.e. because of the budget, not because a global optimality condition was met. Answer: (C).

Why the other options are incorrect.

(A) needs both statements true; II is false.

(B) needs both statements false; I is true.

(D) needs I false and II true; I is true and II is false.

Key takeaway. COBYLA stopped at its evaluation budget; "converged" and "globally optimal" are not the same.

Paragraph for Questions 59 and 60

QAOA with $p = 1$ is applied to the path graph $0 - 1 - 2$ (edges $(0, 1)$ and $(1, 2)$, unit weights, $C_{\max} = 2$) in the notebook's convention, with $\gamma = \pi/2$ and $\beta = -\pi/8$. For a triangle-free graph and $p = 1$ the edge correlator in this convention is

$$\langle Z_u Z_v \rangle = \tfrac{1}{2} \sin 4\beta \, \sin \gamma \, \big(\cos^{d_u - 1} \gamma + \cos^{d_v - 1} \gamma \big),$$

where d_u, d_v are the degrees of the two end nodes.

Q.59 PARAGRAPH BASED · Path graph: $\langle Z_0 Z_1 \rangle$

The value of $\langle Z_0 Z_1 \rangle$ is

(A) ✓ $-\frac{1}{2}$ **(B)** -1 **(C)** $+\frac{1}{2}$ **(D)** 0

Answer: (A) Detailed solution

Correct answer. **(A)** $-\frac{1}{2}$
Concepts required. Evaluating the $p = 1$ triangle-free formula for nodes of different degree.
Relevant formulae. $\langle Z_u Z_v \rangle = \frac{1}{2} \sin 4\beta \sin\gamma (\cos^{d_u - 1}\gamma + \cos^{d_v - 1}\gamma)$; $\cos^0 \gamma = 1$.
Step-by-step explanation. Edge $(0, 1)$: $d_0 = 1$ (node 0 has one neighbour), $d_1 = 2$. With $\gamma = \pi/2$: $\sin\gamma = 1$, $\cos\gamma = 0$, so $\cos^0\gamma = 1$ and $\cos^1\gamma = 0$. With $\beta = -\pi/8$: $\sin 4\beta = \sin(-\pi/2) = -1$.

$$\langle Z_0 Z_1 \rangle = \tfrac{1}{2} \cdot (-1) \cdot 1 \cdot (1 + 0) = -\tfrac{1}{2}.$$

(Note $0^0 = 1$: the term $\cos^{d_u - 1}\gamma$ for a leaf node is 1 for every γ.) By symmetry $\langle Z_1 Z_2 \rangle = -\frac{1}{2}$ too.
Why the other options are incorrect.
(B) -1 uses $\cos^{d-1}\gamma = 1$ for both ends; for node 1 ($d = 2$) it is $\cos\gamma = 0$.
(C) $+\frac{1}{2}$ has the wrong sign: $\sin 4\beta = \sin(-\pi/2) = -1$.
(D) 0 treats $\cos^0\gamma$ as 0 when $\cos\gamma = 0$; but $x^0 = 1$.
Key takeaway. Leaves contribute the factor 1; neighbours with $\cos\gamma = 0$ switch the other term off.

Q.60 PARAGRAPH BASED · Path graph: $\langle C \rangle$, P_{opt}

The expected cut $\langle C \rangle$ and the probability of sampling an optimal split are

(A) 1.5 and 0.3125 **(B)** ✓ 1.5 and 0.625 **(C)** 1.25 and 0.625 **(D)** 2 and 1

Answer: (B) Detailed solution

Correct answer. **(B)** 1.5 and 0.625
Concepts required. Summing edge contributions; probability of the optimal set; complementary optimal strings.
Relevant formulae. $\langle C \rangle = \sum_e \frac{1}{2}(1 - \langle Z_u Z_v \rangle)$; $P_{\text{opt}} = P(010) + P(101)$.
Step-by-step explanation. $\langle C \rangle = \sum_{\text{edges}} \frac{1}{2}(1 - \langle Z_u Z_v \rangle) = 2 \cdot \frac{1}{2}(1 + \frac{1}{2}) = 1.5$, so $r = 1.5/2 = 0.75$. The optimal splits of the path are 010 and 101 (both edges cut). A state-vector calculation gives $P(010) = P(101) = 0.3125$ and 0.0625 for each of the other six strings, so $P_{\text{opt}} = 0.625$. Consistency check: $\langle C \rangle = 2 \cdot 0.625 + 1 \cdot P(C = 1) = 1.25 + 1 \cdot 4 \cdot 0.0625 = 1.5$ (the strings $001, 011, 100, 110$ cut one edge; $000, 111$ cut none).
Why the other options are incorrect.
(A) 0.3125 counts only one of the two optimal strings (010 and its complement 101).
(C) 1.25 adds only the optimal strings' contribution and forgets the four strings that cut one edge.
(D) 2 and 1 would need $\langle Z_u Z_v \rangle = -1$ on both edges; one layer is not enough for the path.
Key takeaway. $r = 0.75$ but $P_{\text{opt}} = 0.625$: a sampler can do better than the average suggests.

SECTION D · Advanced QAOA and Implementation Q.61–80

Q.61 SINGLE-CORRECT MCQ · Light cone of an edge

On a ring graph, the expectation $\langle Z_u Z_v \rangle$ for one edge (u, v) of a depth-p QAOA state depends on at most how many qubits when $p = 3$?

(A) ✓ 8 **(B)** all n qubits **(C)** 7 **(D)** 6

Answer: (A) Detailed solution

Correct answer. **(A)** 8

Concepts required. Heisenberg-picture light cone of a local observable in a local circuit.

Relevant formulae. Support after p layers on a ring: $2 + 2p$ qubits (one new node on each side per cost layer).

Step-by-step explanation. Work in the Heisenberg picture: $\langle Z_u Z_v \rangle = \langle + |^{\otimes n} U^\dagger Z_u Z_v U | + \rangle^{\otimes n}$. Conjugating by a mixer layer (single-qubit gates) keeps the support. Conjugating by a cost layer can only add the neighbours of the current support (an $e^{-i\gamma Z_a Z_b/2}$ acts trivially unless it touches the support). Each layer enlarges the support by one node on each side of the ring: $\{u, v\}$ (2 qubits) $\to$ $4 \to 6 \to 8$ after $p = 3$ layers, i.e. $2p + 2 = 8$ qubits. This *light cone* is why fixed-p QAOA is local, and why large instances of bounded degree can be analysed with small subgraphs.

Why the other options are incorrect.

(B) Gates far from the edge cancel in $U^\dagger Z_u Z_v U$; only the light cone matters.

(C) 7 counts the edge's two nodes plus only one side growing per layer.

(D) 6 is the light cone for $p = 2$.

Key takeaway. At fixed p, QAOA is local: an edge sees only its p-neighbourhood.

Q.62 SINGLE-CORRECT MCQ · Ring at $p = 2$: 5/6

For even rings C_n with $n \geq 2p + 2$, numerical optimisation of the angles gives the best $p = 2$ approximation ratio

(A) 3/4 **(B)** ✓ 5/6 **(C)** 7/8 **(D)** 1

Answer: (B) Detailed solution

Correct answer. **(B)** 5/6

Concepts required. Light-cone independence of n; numerical optimisation of ring QAOA.

Relevant formulae. Ring values (numerical): $r_1 = \frac{3}{4}$, $r_2 = \frac{5}{6}$, $r_3 = \frac{7}{8}$, consistent with $r_p = \frac{2p+1}{2p+2}$.

Step-by-step explanation. For $n \geq 2p + 2$ the light cone of an edge (previous question) is a path of $2p+2$ nodes that does not close around the ring, so every edge sees the same local problem and the ratio does not depend on n. Optimising F_2 numerically for C_6, C_8 and C_{10} (`verify_answers.py`) gives $r = 0.83333 = 5/6$; for $p = 1$ it gives 3/4 (Q.48) and for $p = 3$, 7/8. These values follow the pattern $r_p = \frac{2p+1}{2p+2}$ conjectured by Farhi et al. for the ring: the ratio improves with depth but reaches 1 only as $p \to \infty$ for large rings.

Why the other options are incorrect.

(A) 3/4 is the $p = 1$ value (Q.48).

(C) 7/8 is the $p = 3$ value.

(D) 1 would need $\langle Z_u Z_v \rangle = -1$ for every edge, which a light cone of 6 qubits cannot achieve on a large ring.

Key takeaway. On rings each extra layer closes part of the remaining gap, but never all of it.

Q.63 Single-correct MCQ · Parameter shift vs. finite differences

For the default circuit ($n = 15$, $|E| = 30$, $p = 2$), compare two ways of computing the gradient of F_2 with respect to all $2p$ angles: (i) the exact parameter-shift rule applied to every rotation gate separately (each R_Z and R_X gate is shifted by $\pm\pi/2$ on its own), and (ii) a forward finite-difference gradient as used by L-BFGS-B. The numbers of circuit evaluations are

(A) 8 and 5 **(B)** 180 and 4 **(C)** 90 and 5 **(D)** ✓ 180 and 5

Answer: (D) Detailed solution

Correct answer. (D) 180 and 5

Concepts required. Parameter-shift rule per gate and the chain rule; finite differences.

Relevant formulae. $\partial_\theta f = \frac{1}{2}[f(\theta+\frac{\pi}{2}) - f(\theta-\frac{\pi}{2})]$ per gate; $\partial_{\gamma_l} F = \sum_e w_e \partial_{\theta_e} F$; forward difference: $1 + 2p$ runs.

Step-by-step explanation. **(i)** γ_l enters $|E|$ gates $R_Z(\gamma_l w_e)$; by the chain rule $\partial F/\partial\gamma_l = \sum_e w_e\, \partial F/\partial\theta_e$, and each $\partial F/\partial\theta_e$ needs two runs (shifted by $\pm\pi/2$, Q.16). So $2|E|$ runs per γ_l. Likewise β_l enters n gates $R_X(2\beta_l)$: $2n$ runs. Total $2p(|E| + n) = 2 \cdot 2 \cdot (30 + 15) = 180$. A single two-term shift of γ_l as a whole is *not* exact: its generator H_C has many different eigenvalue gaps. **(ii)** $f(\boldsymbol{\theta})$ once plus one shifted run per parameter: $2p + 1 = 5$ (the notebook's table: "$2p + 1$ circuit runs per gradient"). Finite differences are cheaper but approximate and sensitive to shot noise.

Why the other options are incorrect.

(A) 8 applies a two-term shift to each of the $2p$ angles as a whole, which is not exact because γ_l multiplies all $|E|$ gates.

(B) 4 forgets the evaluation at the unshifted point.

(C) 90 counts one run per gate instead of two shifted runs.

Key takeaway. Exact gradients of shared angles cost 2×(number of gates that use them).

Q.64 Single-correct MCQ · COBYLA `rhobeg`

The notebook optimises the angles with SciPy's `minimize(..., method="COBYLA")` and the options `{"maxiter": 80, "rhobeg": 0.2}`. The option `rhobeg = 0.2` sets

(A) a learning rate multiplying the gradient

(B) the maximum number of iterations divided by 400

(C) the tolerance at which COBYLA declares convergence

(D) ✓ the initial size of the trust region, i.e. the first steps change the angles by about 0.2 rad

Answer: (D) Detailed solution

Correct answer. (D) the initial size of the trust region, i.e. the first steps change the angles by about 0.2 rad

Concepts required. COBYLA: derivative-free trust-region optimisation with linear models.

Relevant formulae. Trust-region radius ρ: starts at `rhobeg`, shrinks to `tol`; each step stays within ρ of the current point.

Step-by-step explanation. COBYLA (Constrained Optimization BY Linear Approximation) is derivative-free: it builds linear models of the objective from function values at a simplex of points and takes steps inside a trust region of radius ρ. `rhobeg` is the starting radius: the first probes move each angle by about 0.2 rad. ρ then shrinks towards `tol`/`rhoend` as the search converges. No gradients (and therefore no learning rate) are used. With only 80 evaluations a too small `rhobeg` would explore little; a too large one would jump across the oscillating landscape.

Why the other options are incorrect.

(A) COBYLA uses no gradients, so there is no learning rate.

(B) `maxiter` is a separate option (80).

(C) The final accuracy is set by `tol` (rhoend), not by `rhobeg`.

Key takeaway. `rhobeg` is the first step size; choose it comparable to the scale of the landscape's features.

Q.65 Single-correct MCQ · Shot noise of the objective

With `OBJECTIVE_SHOTS = 1000`, $\langle H_C \rangle$ is estimated from 1000 shots. If the cut values of the sampled bitstrings have a standard deviation of 2.5, the standard error of the estimated objective is about

(A) 0.0063 **(B)** 0.0025 **(C)** ✓ 0.079 **(D)** 2.5

Answer: (C) Detailed solution

Correct answer. **(C)** 0.079

Concepts required. Standard error of a mean estimated from shots.

Relevant formulae. $\mathrm{SE} = \sigma/\sqrt{S}$.

Step-by-step explanation. The shot estimate is the sample mean of $H_C(z) = \frac{W}{2} - C(z)$; its spread equals that of $C(z)$, $\sigma = 2.5$. Standard error of a mean of S independent samples: $\sigma/\sqrt{S} = 2.5/\sqrt{1000} = 2.5/31.6 \approx 0.079$. Differences between neighbouring optimizer steps near the optimum are often of this size or smaller, so the optimizer starts to chase noise; that is why the notebook warns that shot-based objectives "make optimisation harder".

Why the other options are incorrect.

(A) $0.0063 = \sigma^2/S$ is the variance of the mean, not its standard error.

(B) $0.0025 = \sigma/S$ divides by S instead of $\sqrt{S}$.

(D) 2.5 is the spread of single shots, not of their mean.

Key takeaway. 1000 shots give an objective accurate to about ± 0.08 — noise the optimizer must live with.

Q.66 Single-correct MCQ · Brute-force memory

For $n = 22$ (`BRUTE_FORCE_MAX_NODES`), the notebook's `brute_force_maxcut` creates the arrays `index = np.arange(2**N, dtype=np.int64)` and `values = np.zeros(2**N)`. Together they occupy about

(A) 34 MB **(B)** ✓ 67 MB **(C)** 0.5 MB **(D)** 4.2 MB

Answer: (B) Detailed solution

Correct answer. **(B)** 67 MB

Concepts required. Memory of NumPy arrays: entries × bytes per entry.

Relevant formulae. $2^{22} = 4\,194\,304$; int64 and float64: 8 bytes; total $2 \times 8 \times 2^{22}$ bytes.

Step-by-step explanation. $2^{22} = 4\,194\,304$ entries per array. `int64` and `float64` both use 8 bytes: $2 \times 8 \times 4\,194\,304 = 67\,108\,864$ bytes ≈ 67 MB (plus temporaries of the same size created per edge while evaluating `(index » a) ^ (index » b)`). 34 MB is one array; 4.2 MB is the number of *entries* read as bytes. Each additional node doubles both memory and time, which is why the exact reference stops at 22 nodes.

Why the other options are incorrect.

(A) 34 MB is one of the two arrays.

(C) 0.5 MB is the 15-qubit state vector, unrelated.

(D) 4.2 MB is the number of entries (4.2 million), not bytes.

Key takeaway. Brute force costs $O(2^n)$ memory and $O(|E|\,2^n)$ time: fine at 22 nodes, hopeless at 40.

Q.67 SINGLE-CORRECT MCQ · Multi-GPU state vector

A 36-qubit QAOA state vector is simulated in fp64. If it is split evenly over a power-of-two number of 80 GB GPUs (as the multi-GPU state-vector target does), ignoring workspace, the minimum number of GPUs is

(A) 8 **(B)** 14 **(C)** 32 **(D)** ✓ 16

Answer: (D) **Detailed solution**

Correct answer. **(D)** 16
Concepts required. Distributing a state vector over a power-of-two number of devices.
Relevant formulae. memory $= 16 \times 2^{36}$ B ≈ 1.10 TB; per GPU $= 1.10$ TB$/2^k \le 80$ GB $\Rightarrow k \ge 4$.
Step-by-step explanation. Memory: $16 \times 2^{36} = 1.0995 \times 10^{12}$ bytes ≈ 1.10 TB. Dividing by 80 GB: 13.7, so at least 14 GPUs by memory alone. The state is distributed in power-of-two slices (2^k GPUs each holding 2^{36-k} amplitudes): $2^3 = 8$ GPUs would need 137 GB each (too much), $2^4 = 16$ GPUs need 68.7 GB each (fits). So 16 GPUs, launched with one MPI rank per GPU (`BACKEND = "multi-gpu"`, `mpiexec`). These GPU paths are implemented in the notebook but were not executed on the CPU-only test machine.
Why the other options are incorrect.
(A) 8 GPUs would need 137 GB each.
(B) 14 GPUs suffice by memory, but the state is split into power-of-two slices.
(C) 32 is more than necessary; 16 already fit.
Key takeaway. Each extra qubit doubles the number of GPUs needed.

Q.68 MATCHING LISTS · Backend / method → target

Match each setting of `BACKEND` and `SIMULATION_METHOD` in `QAOA.ipynb` (List-I) with what the notebook's `resolve_backend` does (List-II).

List-I		List-II	
(P)	`"cpu"` + `"statevector"`	**(1)**	target `tensornet-mps`
(Q)	`"cpu"` + `"density_matrix"`	**(2)**	stops with an error: the MPS simulator needs an NVIDIA GPU
(R)	`"gpu"` + `"mps"`	**(3)**	target `qpp-cpu`
(S)	`"cpu"` + `"mps"`	**(4)**	target `density-matrix-cpu`

(A) P→4, Q→3, R→1, S→2 **(B)** P→3, Q→1, R→4, S→2
(C) P→3, Q→4, R→2, S→1 **(D)** ✓ P→3, Q→4, R→1, S→2

Answer: (D) **Detailed solution**

Correct answer. **(D)** P→3, Q→4, R→1, S→2
Concepts required. The notebook's backend table and its explicit error for unsupported combinations.
Relevant formulae. `_TARGET_TABLE[(backend, method)]` → CUDA-Q target; missing keys raise `ValueError` with `_UNSUPPORTED_HINTS`.
Step-by-step explanation. From `_TARGET_TABLE` and `_UNSUPPORTED_HINTS` in the shared utilities (QAOA allows the methods `statevector`, `mps`, `tensornet`, `density_matrix`):

- P: `("cpu", "statevector")` → `qpp-cpu` (OpenMP state vector, fp64) (3) — the target of the recorded run.
- Q: `("cpu", "density_matrix")` → `density-matrix-cpu` (4); memory 16×4^n bytes.

- R: `("gpu", "mps")` → `tensornet-mps` (1).
- S: `("cpu", "mps")` is not in the table; the hint says "CUDA-Q's MPS simulator (target 'tensornet-mps') needs an NVIDIA GPU" and a `ValueError` is raised (2).

Why the other options are incorrect.

(A) P→4, Q→3 swaps the two CPU targets; `density_matrix` selects `density-matrix-cpu`.

(B) Q→1 maps the density matrix to an MPS target; these are different simulation methods.

(C) R→2, S→1 reverses which MPS combination works: MPS (`tensornet-mps`) needs a GPU.

Key takeaway. CPU: `qpp-cpu` / `density-matrix-cpu`; GPU: `nvidia`, `tensornet-mps`, `tensornet`.

Q.69 Single-correct MCQ · When MPS is efficient

For which instance is a matrix-product-state (MPS) simulation of a depth-p QAOA circuit most likely to be efficient (small bond dimension), with the qubits ordered along a line $0, 1, \ldots, n-1$?

(A) Any graph, because QAOA states are always product states before measurement

(B) A complete graph K_n: its symmetry makes the MPS exact with bond dimension 1

(C) ✓ A ring C_n with small p: every cut of the line is crossed by only 2 edges

(D) A dense random graph with edge probability 0.5: more edges average out the entanglement

Answer: (C) Detailed solution

Correct answer. (C) A ring C_n with small p: every cut of the line is crossed by only 2 edges

Concepts required. Entanglement across a bipartition bounds the MPS bond dimension.

Relevant formulae. $\chi_k \le 2^{p\, m_k}$, m_k = number of edges crossing the cut between qubits k and $k+1$.

Step-by-step explanation. An MPS across a cut between qubits k and $k+1$ needs a bond dimension at least the Schmidt rank there. Each cost gate $e^{-i\theta Z_a Z_b/2}$ that straddles the cut can at most double that rank, so after p layers $\chi \le 2^{p \cdot m_k}$, where m_k is the number of edges crossing the cut. For a ring in natural order $m_k = 2$ for every cut, so $\chi \le 4^p$: small for small p. Dense random graphs have $m_k = \Theta(n^2)$ crossings: the bond dimension can grow exponentially. K_n is not product-like either: its cost layer entangles all pairs (Q.28). The cost layer creates entanglement, so QAOA states are not product states.

Why the other options are incorrect.

(A) The cost layer entangles qubits (Q.28); QAOA states are generally entangled.

(B) K_n has $\approx n^2/4$ edges across the middle cut; its QAOA state is not a product state.

(D) More edges mean more entangling gates across each cut, hence *larger* bond dimension.

Key takeaway. MPS likes 1D-like graphs at low depth; dense graphs need state vectors or tensor networks.

Q.70 Assertion–Reason · Shot-based objective on hardware

Assertion (A): When `BACKEND = "hardware"`, the notebook estimates the objective from shots even if `OBJECTIVE_SHOTS = None`.

Reason (R): Hardware (and hardware-emulation) targets only return measurement samples; they cannot give a state vector or an exact expectation value.

(A) ✓ Both Assertion (A) and Reason (R) are true, and R is the correct explanation of A

(B) Both A and R are true, but R is *not* the correct explanation of A

(C) A is true, but R is false

(D) A is false, but R is true

Answer: (A) Detailed solution

Correct answer. **(A)** Both Assertion (A) and Reason (R) are true, and R is the correct explanation of A

Concepts required. Hardware returns samples only; the code switches to shot-based objectives automatically.

Relevant formulae. $\widehat{\langle H_C \rangle} = \frac{1}{S}\sum_{\text{shots}} H_C(z)$, $S =$ `SHOTS` on hardware.

Step-by-step explanation. **A is true:** the code sets `objective_shots = OBJECTIVE_SHOTS if OBJECTIVE_SHOTS is not None else (SHOTS if backend_cfg["device"] == "hardware" else None)`, so on hardware $\langle H_C \rangle$ is estimated with `SHOTS` $= 4000$ shots per evaluation. **R is true and is the reason:** a QPU only produces bitstrings; exact expectation values exist only in simulators (README: "Hardware targets support sampling only"). Answer: (A).

Why the other options are incorrect.

(B) needs A and R both true but R not explaining A; R is exactly why the code makes this choice.

(C) needs A true and R false; R is true.

(D) needs A false and R true; A is true.

Key takeaway. On hardware every objective value is a noisy shot estimate.

Q.71 Single-correct MCQ · Depolarising-noise model

A simple global-depolarising noise model treats the noisy QAOA output as a mixture: with probability $F = (1-\varepsilon)^{N_{\text{CNOT}}}$ the ideal state, otherwise a uniformly random bitstring. For the default circuit ($N_{\text{CNOT}} = 120$, ideal $\langle C \rangle = 20.53$, $W/2 = 15$) and $\varepsilon = 0.005$ per CNOT, the predicted noisy $\langle C \rangle$ is about

(A) 17.2 **(B)** 15.0 **(C)** ✓ 18.0 **(D)** 11.3

Answer: (C) Detailed solution

Correct answer. **(C)** 18.0

Concepts required. Global depolarising model: mixture of the ideal output with the uniform distribution.

Relevant formulae. $F = (1-\varepsilon)^{N_{\text{CNOT}}}$; $\langle C \rangle_{\text{noisy}} = F\langle C \rangle_{\text{ideal}} + (1-F)\frac{W}{2}$.

Step-by-step explanation. $F = 0.995^{120} = e^{120 \ln 0.995} = e^{-0.6015} \approx 0.548$. Mixture: $\langle C \rangle_{\text{noisy}} = F\,\langle C \rangle_{\text{ideal}} + (1-F)\,\frac{W}{2} = 0.548 \cdot 20.53 + 0.452 \cdot 15 = 11.25 + 6.78 \approx 18.0$. 11.3 forgets the random part (a random bitstring still cuts $W/2$ on average); 17.2 uses the linearisation $F \approx 1 - 120\varepsilon = 0.4$, which is poor when 120ε is not small. The model is illustrative only (real noise is not exactly global depolarising), but it shows how noise pulls QAOA towards the random baseline and limits useful depth.

Why the other options are incorrect.

(A) 17.2 uses $F \approx 1 - 120\varepsilon = 0.40$; the exact $F = 0.548$.

(B) 15.0 is the fully depolarised limit $F = 0$.

(D) 11.3 keeps only $F\langle C\rangle_{\text{ideal}}$; the random part still contributes $W/2$ per shot.
Key takeaway. Noise pulls $\langle C\rangle$ towards $W/2$ exponentially in the number of two-qubit gates.

Q.72 MATCHING LISTS · QAOA variants

Match each QAOA variant (List-I) with its key idea (List-II).

List-I		List-II	
(P)	warm-start QAOA	**(1)**	problem-specific mixers (e.g. XY mixers) that keep the state inside the feasible subspace of a constrained problem
(Q)	ADAPT-QAOA	**(2)**	use measured correlations $\langle Z_iZ_j\rangle$ to fix and eliminate variables, then solve the smaller instance
(R)	Quantum Alternating Operator Ansatz (Hadfield et al.)	**(3)**	start from a state biased towards a classical (e.g. relaxed/SDP) solution instead of $\lvert+\rangle^{\otimes n}$
(S)	recursive QAOA (RQAOA)	**(4)**	grow the mixer layer by layer from an operator pool, choosing the operator with the largest energy gradient

(A) P→3, Q→1, R→4, S→2
(B) P→3, Q→4, R→2, S→1
(C) ✓ P→3, Q→4, R→1, S→2
(D) P→4, Q→3, R→1, S→2

Answer: (C) **Detailed solution**

Correct answer. **(C)** P→3, Q→4, R→1, S→2
Concepts required. Main families of QAOA extensions.
Relevant formulae. Warm start: $|\psi_0\rangle = \bigotimes_i R_Y(\theta_i)\,|0\rangle$ from a relaxed solution; RQAOA: fix $z_i = \text{sign}\langle Z_iZ_j\rangle\, z_j$ for the largest $|\langle Z_iZ_j\rangle|$.
Step-by-step explanation.

- P: warm-start QAOA (Egger, Mareček, Woerner 2021) changes the initial state (and matching mixer) using a classical relaxation (3).
- Q: ADAPT-QAOA (Zhu et al. 2022) selects mixers adaptively from a pool by gradient (4).
- R: the Quantum Alternating Operator Ansatz (Hadfield et al. 2019) generalises the mixer to respect constraints (1).
- S: RQAOA (Bravyi et al. 2020) eliminates variables using the strongest correlations, recursively (2).

None of these is implemented in `QAOA.ipynb`, which uses the standard X mixer and $\lvert+\rangle^{\otimes n}$.
Why the other options are incorrect.

(A) Q→1, R→4: feasibility-preserving mixers are Hadfield's idea; ADAPT grows mixers from a pool.

(B) R→2, S→1 swaps the constrained-mixer ansatz and RQAOA.

(D) P→4, Q→3 swaps warm start and ADAPT: ADAPT changes the mixer, warm start changes the initial state.

Key takeaway. Variants change the start state, the mixer, or the outer loop — the notebook implements standard QAOA.

Q.73 Single-correct MCQ · INTERP initialisation

The INTERP initialisation (Zhou et al. 2020) builds depth-$(p+1)$ starting angles from optimised depth-p angles by $\gamma_i^{(p+1)} = \frac{i-1}{p}\,\gamma_{i-1}^{(p)} + \frac{p-i+1}{p}\,\gamma_i^{(p)}$ for $i = 1, \ldots, p+1$ (with $\gamma_0^{(p)} = \gamma_{p+1}^{(p)} = 0$). From $\boldsymbol{\gamma}^{(2)} = (0.4, 0.8)$ it gives $\boldsymbol{\gamma}^{(3)} =$

(A) ✓ $(0.4,\ 0.6,\ 0.8)$
(B) $(0.27,\ 0.53,\ 0.8)$
(C) $(0.4,\ 0.8,\ 0.8)$
(D) $(0.4,\ 0.8,\ 0)$

Answer: (A) Detailed solution

Correct answer. **(A)** $(0.4,\ 0.6,\ 0.8)$
Concepts required. INTERP: linear interpolation of an optimised schedule onto $p+1$ points.
Relevant formulae. $\gamma_i^{(p+1)} = \frac{i-1}{p}\gamma_{i-1}^{(p)} + \frac{p-i+1}{p}\gamma_i^{(p)}$, $\gamma_0 = \gamma_{p+1} = 0$.
Step-by-step explanation. $p = 2$. $i = 1$: $\frac{0}{2}\gamma_0 + \frac{2}{2}\gamma_1 = 0.4$. $i = 2$: $\frac{1}{2}\gamma_1 + \frac{1}{2}\gamma_2 = \frac{1}{2}(0.4 + 0.8) = 0.6$. $i = 3$: $\frac{2}{2}\gamma_2 + \frac{0}{2}\gamma_3 = 0.8$. So $(0.4, 0.6, 0.8)$: the optimised schedule is linearly interpolated onto one more point, keeping its smooth shape. This is a general strategy from the literature; the notebook itself starts every depth from its fixed ramp.
Why the other options are incorrect.
(B) $(0.27, 0.53, 0.8)$ is a linear ramp from 0, which discards the optimised first angle.
(C) $(0.4, 0.8, 0.8)$ copies the last angle; INTERP gives the middle point 0.6.
(D) $(0.4, 0.8, 0)$ appends a zero layer — equivalent to depth p, a valid but different start.
Key takeaway. Good angles at depth p are smooth schedules; interpolate them to start depth $p+1$.

Q.74 Statement I / II · Heuristic reference: upper bound on r

For graphs with more nodes than `BRUTE_FORCE_MAX_NODES`, the notebook replaces the exact optimum by the best cut found by `local_search_maxcut` and still reports $r = \langle C \rangle / \texttt{max_cut}$.

Statement I: In that case the reported `max_cut` is guaranteed to be at least the true $C_{\max}$.

Statement II: In that case the reported ratio is at least the true ratio $\langle C \rangle / C_{\max}$.

(A) Both Statement I and Statement II are true
(B) Both Statement I and Statement II are false
(C) Statement I is true, but Statement II is false
(D) ✓ Statement I is false, but Statement II is true

Answer: (D) Detailed solution

Correct answer. **(D)** Statement I is false, but Statement II is true
Concepts required. A heuristic lower bound on $C_{\max}$ makes the reported ratio an upper bound.
Relevant formulae. $\texttt{max_cut}_{\text{heur}} \le C_{\max} \Rightarrow \langle C \rangle / \texttt{max_cut}_{\text{heur}} \ge \langle C \rangle / C_{\max}$.
Step-by-step explanation. The heuristic returns the cut of an actual split, so `max_cut` $\le C_{\max}$ (it can only miss the optimum, never exceed it). **I is false** (the inequality goes the other way). **II is true:** dividing the same positive $\langle C \rangle$ by a smaller (or equal) number gives a larger (or equal) value: $\langle C \rangle / \texttt{max_cut} \ge \langle C \rangle / C_{\max}$. The reported ratio is an *upper bound* on the true one, which is why the notebook labels the reference "NOT guaranteed optimal". Answer: (D).
Why the other options are incorrect.
(A) needs both statements true; I is false.
(B) needs both statements false; II is true.
(C) needs I true and II false; I is false and II is true.
Key takeaway. Without an exact reference, the reported ratio is optimistic.

Q.75 SINGLE-CORRECT MCQ · Recorded timing per gate

In the recorded run, the quantum execution time was 40.01 s for the 80 objective evaluations (`cudaq.observe` calls) of the 225-gate circuit on 15 qubits. The average time per evaluation and per gate are about

(A) 0.010 s and 44 μs
(B) 0.18 s and 0.8 ms
(C) ✓ 0.50 s and 2.2 ms
(D) 40 s and 0.18 s

Answer: (C) Detailed solution

Correct answer. **(C)** 0.50 s and 2.2 ms
Concepts required. Turning recorded timings into per-evaluation and per-gate costs; comparing with the calibration.
Relevant formulae. $t_{\text{eval}} = t_{\text{exec}}/N_{\text{eval}}$; $t_{\text{gate}} = t_{\text{eval}}/N_{\text{gates}}$.
Step-by-step explanation. Per evaluation: $40.01/80 \approx 0.50$ s. Per gate: $0.50/225 \approx 2.2 \times 10^{-3}$ s = 2.2 ms. This matches the notebook's own calibration printed before execution: "1.0921 ms/gate at 14 qubits, extrapolated $\times 2^1$ to 15 qubits" → 2.18 ms/gate, and the estimate $\sim$ 39.8 s versus 40.84 s measured for the whole cell. (0.010 s divides by the 4000 shots, which play no role in exact `observe` calls.)
Why the other options are incorrect.
(A) Dividing by 4000 shots is wrong: the exact `observe` calls use no shots.
(B) 0.18 s divides the total by the number of gates instead of the number of evaluations.
(D) 40 s is the total, not per evaluation.
Key takeaway. About 2.2 ms per gate at 15 qubits on the 4-core CPU, matching the notebook's own calibration.

Q.76 SINGLE-CORRECT MCQ · Run-time scaling to 25 nodes

The notebook estimates run time as (number of evaluations) × (gates per circuit) × (time per gate), with the time per gate scaling as 2^n for a state vector. Going from the default ($n = 15$, $|E| = 30$, $p = 2$) to $n = 25$ with $|E| = 90$ edges at the same p and evaluation budget, the run time grows by a factor of about

(A) 2.9×10^6 **(B)** ✓ 2.8×10^3 **(C)** 2.7 **(D)** 1.0×10^3

Answer: (B) Detailed solution

Correct answer. **(B)** 2.8×10^3
Concepts required. The notebook's runtime model: gates × per-gate cost $\propto 2^n$.
Relevant formulae. $\frac{T_{25}}{T_{15}} = \frac{25+2(3\cdot 90+25)}{15+2(3\cdot 30+15)} \times 2^{10} = \frac{615}{225} \times 1024$.
Step-by-step explanation. Gates: $n + p(3|E| + n)$: $15 + 2(90 + 15) = 225$ before, $25 + 2(270 + 25) = 615$ after; ratio $615/225 = 2.73$. Time per gate: $2^{25-15} = 1024$ times larger. Total factor $2.73 \times 1024 \approx 2.8 \times 10^3$, i.e. the recorded ≈ 40 s would become ≈ 31 hours on the same 4-core CPU — the reason the notebook suggests a GPU for 24–30 nodes. 1.0×10^3 ignores the extra gates; 2.7 ignores the exponential state size; 2.9×10^6 uses the $4^{\Delta n}$ scaling of a density matrix. (The model is the notebook's rough estimate, quoted as accurate to about $\pm 3\times$.)
Why the other options are incorrect.
(A) 2.9×10^6 uses $4^{\Delta n}$, the density-matrix scaling; this is a state-vector run.
(C) 2.7 is only the gate ratio; it ignores that each gate acts on a 1024× larger state.
(D) 1.0×10^3 is only the 2^{10} factor; the circuit also has 2.73× more gates.
Key takeaway. Ten more qubits cost three orders of magnitude on a state-vector simulator.

Q.77 SINGLE-CORRECT MCQ · Quantum advantage: state of knowledge

Which statement about QAOA for Max-Cut reflects the current state of knowledge?

(A) ✓ No quantum advantage has been demonstrated: the classical Goemans–Williamson algorithm guarantees 0.878 in polynomial time, and fixed-depth QAOA is limited by locality

(B) The recorded result $r = 0.82$ on 15 nodes proves a quantum speed-up

(C) QAOA solves Max-Cut exactly in polynomial time for any fixed p

(D) QAOA with $p = 1$ beats Goemans–Williamson on every 3-regular graph

Answer: (A) Detailed solution

Correct answer. **(A)** No quantum advantage has been demonstrated: the classical Goemans–Williamson algorithm guarantees 0.878 in polynomial time, and fixed-depth QAOA is limited by locality

Concepts required. Known guarantees and limitations of QAOA versus classical algorithms.

Relevant formulae. GW: 0.878 (polynomial, classical); QAOA $p = 1$, 3-regular: 0.6924.

Step-by-step explanation. Known facts: $p = 1$ QAOA guarantees 0.6924 on 3-regular graphs (Q.49), below the Goemans–Williamson 0.878. Locality (light cone, Q.61) limits any fixed-p QAOA: e.g. Bravyi et al. (2020) showed that on certain bipartite regular graphs low-depth QAOA stays far from the optimum. Max-Cut is NP-hard, so no polynomial exact algorithm is expected. A 15-node simulation that a laptop brute-forces in 0.01 s cannot show a speed-up; QAOA's value is as a well-understood, tunable heuristic and a benchmark.

Why the other options are incorrect.

(B) A 15-node instance is brute-forced in 0.01 s; small simulations cannot show a speed-up.

(C) No fixed-p QAOA is known to solve an NP-hard problem exactly; that would imply surprising complexity results.

(D) $0.6924 < 0.878$: $p = 1$ QAOA has the weaker worst-case guarantee.

Key takeaway. Treat QAOA as a heuristic with honest benchmarks, not as a proven speed-up.

Q.78 SINGLE-CORRECT MCQ · Custom weighted edges

With `GRAPH = "custom"` and `CUSTOM_EDGES = [(0, 1, 1.0), (1, 2, 2.5), (0, 2, -1.0)]` (3 nodes), the maximum cut and the random-split baseline printed by the notebook are

(A) 3.5 and 2.25 **(B)** ✓ 3.5 and 1.25 **(C)** 2.5 and 1.25 **(D)** 4.5 and 2.25

Answer: (B) Detailed solution

Correct answer. **(B)** 3.5 and 1.25

Concepts required. Weighted (including negative) Max-Cut; the baseline uses the signed total weight.

Relevant formulae. $C(z) = \sum w_{ij}[z_i \neq z_j]$; baseline $W/2$ with $W = \sum w_{ij}$ (signed).

Step-by-step explanation. Negative weights are allowed ("weights may be any real numbers"); a cut edge with weight -1 *lowers* the cut. The non-trivial partitions isolate one node: $\{0\}$: edges $(0,1),(0,2)$: $1 - 1 = 0$. $\{1\}$: $(0,1),(1,2)$: $1 + 2.5 = 3.5$. $\{2\}$: $(1,2),(0,2)$: $2.5 - 1 = 1.5$. So $C_{\max} = 3.5$ (strings 010 and 101). Baseline $W/2$ with $W = 1 + 2.5 - 1 = 2.5$: 1.25. The rest of the pipeline is unchanged: the edge gets `rz(gamma * w)` with $w = -1$ and H_C the term $-0.5\, Z_0 Z_2$.

Why the other options are incorrect.

(A) The baseline uses the signed $W = 2.5$, not $\sum |w| = 4.5$.

(C) 2.5 is W, not the maximum cut (isolating node 1 gives 3.5).

(D) 4.5 and 2.25 use $|w|$, as if the negative edge also counted positively.

Key takeaway. Negative weights reward keeping nodes together; check every split, not just the obvious ones.

Paragraph for Questions 79 and 80

Previously recorded run of `QAOA.ipynb` (default settings, CUDA-Q 0.16.0, target `qpp-cpu`): 15 nodes, 30 unit edges ($W = 30$), exact optimum $C_{\max} = 25$ with the four optimal bitstrings listed in A14, $p = 2$, COBYLA with 80 evaluations, best expected cut $\langle C \rangle = 20.5336$, 4000 shots. The four most frequent bitstrings were all optimal: `110010001001101` (68 shots), `001101110110010` (67), `110010001011101` (56), `001101110100010` (49); in total 240 shots were optimal.

Q.79 PARAGRAPH BASED · Recorded run: partition probability

Estimated from the recorded counts, the probability of sampling the partition described by `110010001001101` (either this bitstring or its complement) is

(A) ✓ 0.0338 **(B)** 0.0600 **(C)** 0.0170 **(D)** 0.0263

Answer: (A) Detailed solution

Correct answer. **(A)** 0.0338

Concepts required. A partition corresponds to a bitstring and its complement; estimating probabilities from counts.

Relevant formulae. $\hat{P}(\{z, \bar{z}\}) = (n_z + n_{\bar{z}})/S$.

Step-by-step explanation. The complement of `110010001001101` is `001101110110010` (flip every bit), which is the second most frequent string. Both describe the same partition, so the estimate is $(68 + 67)/4000 = 135/4000 = 0.03375 \approx 0.0338$. The other optimal partition (`110010001011101` / `001101110100010`) has $(56 + 49)/4000 = 0.02625$; together $0.03375 + 0.02625 = 0.06 = P_{\text{opt}}$. The nearly equal counts within each pair (68 vs 67, 56 vs 49) reflect the exact symmetry $|\langle z|\psi\rangle| = |\langle \bar{z}|\psi\rangle|$ of the QAOA state, up to shot noise.

Why the other options are incorrect.

(B) 0.0600 is the probability of *both* optimal partitions together.

(C) $0.0170 = 68/4000$ counts only one of the two bitstrings of the partition.

(D) 0.0263 is the other optimal partition.

Key takeaway. Add z and $\bar{z}$ to get a partition's probability; the two counts should agree up to shot noise.

Q.80 PARAGRAPH BASED · Recorded run: improvement over random

A stricter figure of merit measures the improvement over random guessing: $\eta = \dfrac{\langle C \rangle - W/2}{C_{\max} - W/2}$.

For the recorded run, η is

(A) 0.600 **(B)** ✓ 0.553 **(C)** 0.221 **(D)** 0.821

Answer: (B) Detailed solution

Correct answer. **(B)** 0.553

Concepts required. Normalising performance relative to random guessing.

Relevant formulae. $\eta = \dfrac{\langle C \rangle - W/2}{C_{\max} - W/2}$; $r_{\text{random}} = \dfrac{W/2}{C_{\max}}$.

Step-by-step explanation. $\eta = \dfrac{20.5336 - 15}{25 - 15} = \dfrac{5.5336}{10} \approx 0.553$. The usual ratio $r = 20.5336/25 = 0.821$ gives random guessing already $15/25 = 0.6$, so r looks better than the actual progress. $\eta = 0$ for random guessing and 1 for the optimum: QAOA at $p = 2$ closed about 55 % of the gap between random guessing and the optimum.

Why the other options are incorrect.

(A) 0.600 is the ratio of random guessing.

(C) $0.221 = 0.821 - 0.6$ subtracts ratios instead of normalising by the remaining gap.

(D) 0.821 is the ordinary approximation ratio r.

Key takeaway. Quote r together with the random baseline; η shows the real progress.

SILICO FELLER